

Highlights

CTHR: Constraint-Traceable Hierarchical Rule Compilation for Auditable KG-Driven Optimization

Longtao Zhu, Jun Li, Jiahong Xie, Ningwei Bai, Hai Jiang, Yulong Ji

- CTHR compiles source-KG rules into auditable feasible cells with rule-level and document-level provenance.
- CTHR resolves applicability, dependency, exclusion, override, precedence, and parameter propagation before optimization.
- A 200-task aviation/building benchmark evaluates semantic validity, solver reuse, and audit traceability.

CTHR: Constraint-Traceable Hierarchical Rule Compilation for Auditable KG-Driven Optimization

Longtao Zhu^a, Jun Li^b, Jiahong Xie^c, Ningwei Bai^e, Hai Jiang^d and Yulong Ji^{d,*}

^aCollege of Computer Science, Sichuan University, Chengdu, China

^bSystem of Systems and Artificial Intelligence Key Laboratory of SICHUAN Province, Chengdu, China

^cAVIC Chengdu Aircraft Design and Research Institute, Chengdu, China

^dSchool of Aeronautics and Astronautics, Sichuan University, Chengdu, China

^eDepartment of Computer Science, University College London (UCL), London, United Kingdom

ARTICLE INFO

Keywords:

Knowledge-based systems
Knowledge graphs
Defeasible reasoning
Constraint compilation
Auditable optimization

ABSTRACT

In safety-critical domains like flight procedure and building design, knowledge-based decision support systems must return decisions that are numerically feasible, consistent with source rules, and traceable to evidence. Although design specifications can be organized as knowledge graphs (KGs), KGs do not directly provide optimizer-ready feasible regions with provenance. To address this, we propose CTHR (Constraint-Traceable Hierarchical Rule Compilation), a provenance-preserving symbolic modeling framework that transforms KG rules into auditable optimization feasible regions. For a given scenario, CTHR resolves six rule relations—applicability, dependency, mutual exclusion, override, precedence, and parameter propagation—to construct valid rule structures and compile them into exact feasible regions annotated with rule- and document-level provenance. These regions are consumed by downstream constrained optimizers, yielding decisions with certificates tracing back to KG evidence. Theoretical analysis shows CTHR preserves the constraint scope induced by source-rule semantics without admitting prohibited decisions or excluding valid compliance paths. Experiments on 200 tasks from flight procedure and building design show that flat-rule modeling often produces source-semantically invalid decisions despite high rule recall. On the aviation benchmark, the flat baseline returns invalid decisions for all tasks, whereas CTHR-style SCIP reduces invalid cases from 150 to 17 and reaches 88.7% Sem-CSR. On the architecture benchmark, CTHR-style backends reduce invalid cases from 11 to 2, reaching 96.0% Sem-CSR. Solver-backend experiments show that HiGHS, CP-SAT/OR-Tools, and SCIP successfully optimize over CTHR feasible cells while preserving audit certificates. These results validate CTHR as a provenance-preserving KG-to-rule-to-feasible-cell modeling layer rather than a new optimizer. Code and data are available at <https://github.com/Zhulongtao6/CTHR-Constraint-Traceable-Hierarchical-Rule-Compilation-for-Auditable-KG-Driven-Optimization>. git.

1. Introduction

Knowledge-based decision systems are increasingly being deployed in high-assurance industrial processes, such as flight procedure design and building design. In these domains, a common practice is to organize requirements from regulatory documents, standards, and operating manuals as knowledge graphs (KGs) [1, 2], where entities, relations, source-document links, and rule annotations jointly represent domain knowledge. Such KGs support rule retrieval, association among related requirements, and provenance tracking. However, they do not by themselves transform the rules retrieved for a concrete decision problem into the feasible region required for optimization, nor do they directly support tracing an optimized decision back to the source rules that justify it.

Once domain knowledge has been encoded as a KG, it still needs to be converted into an operational decision-making

mechanism. In practical settings, this conversion involves mapping KG evidence into a structured rule library, selecting the rules applicable to a concrete scenario, compiling the resulting constraints, and solving an optimization problem. At present, this process is largely carried out by experts who understand both domain regulations and optimization modeling. Moreover, real-world KGs do not consist of merely flat rules. They often contain exception rules, cross-document precedence, mutually exclusive alternatives, and scenario-dependent conditions that can simultaneously affect multiple requirements. Therefore, transforming symbolic knowledge into a feasible decision space is a core knowledge-engineering problem rather than a simple preprocessing step.

Most constrained learning and optimization methods assume a flat rule-to-constraint mapping: each active rule contributes an independent inequality, and the conjunction of these inequalities is treated as the feasible set. Source KGs for technical specifications do not operate in this way. Exception rules may override base rules for specific procedure categories; cross-document precedence determines which source should govern when documents conflict; and alternative templates may correspond to distinct compliance

*Corresponding author

✉ zhulongtao@scu.edu.cn (L. Zhu); ningwei.bai.23@ucl.ac.uk (N. Bai); jianghai@scu.edu.cn (H. Jiang); jyl@scu.edu.cn (Y. Ji)
ORCID(s): 0000-0003-3116-5759 (Y. Ji)

paths rather than requirements that must be merged. For example, in building-code compliance, whether a building is equipped with an automatic sprinkler system may change maximum travel-distance limits, exit-width thresholds, and fire-separation requirements. If the modeling layer ignores these rule interactions, an optimizer may solve a region that appears numerically feasible but is inconsistent with the source-rule semantics. It may also lose the provenance required to explain why the returned decision is compliant.

This paper focuses on the modeling layer between a source KG and an optimizer. Pure neural constrained-optimization approaches [3, 4] can improve objective performance under learned or given constraints, but they usually assume that the feasible set is already correct and do not preserve rule-level provenance. Pure symbolic reasoning and SMT-based checking methods [5] can verify compliance, but they do not directly provide a source-linked feasible region that can be reused by downstream optimizers across decision queries. Knowledge-based decision systems therefore require a modeling method that can transform source-KG rule semantics into optimization constraints for a concrete problem while preserving the feasible decision scope and retaining audit evidence.

To address this question, this paper proposes CTHR (Constraint-Traceable Hierarchical Rule Compilation), a provenance-preserving KG-to-rule-to-feasible-cell modeling layer for KG-driven optimization. In CTHR, source-KG evidence is first mapped into a rule library. Then, for each concrete scenario, CTHR resolves rule applicability, dependencies, mutual exclusions, overrides, and precedence relations, together with parameter propagation; compiles the remaining valid rule structures into auditable feasible cells; and preserves source-document provenance for each feasible cell. The six rule relations are not introduced as new logical primitives. The core contribution is their joint use as a source-linked compilation target: CTHR turns scenario-specific KG rules into solver-consumable feasible cells without losing alternative compliance paths or provenance. These feasible cells can then be passed to an exact solver, a classical constrained optimizer, or a multi-objective search method. CTHR is therefore not proposed as a new optimizer; its role is to define the source-consistent feasible region and the traceability interface that downstream solvers operate on. The system returns not only a numerical decision, but also a source-linked audit record that allows domain experts to trace the decision back to the active rules and source documents that support it.

The main contributions of this paper are summarized as follows:

- **Joint compilation of KG rule interactions.** This paper defines how scenario applicability, dependencies, mutual exclusions, overrides, precedence, and parameter propagation are jointly grounded for a concrete design scenario. CTHR resolves these source-linked relations before optimization and represents alternative compliance paths as distinct valid rule structures rather than flattening them into one constraint set.

- **Source-semantics-preserving feasible-cell compilation.** The framework compiles each valid rule structure into an auditable feasible cell and proves that the union of feasible cells preserves the feasible region induced by the source-rule semantics. Each cell retains rule-level and document-level provenance, so the feasible region is both executable and source traceable.
- **Solver-agnostic optimization interface.** CTHR defines the auditable feasible cells and exposes them to downstream backends including the default optimizer, ASP/clingo, CP-SAT/OR-Tools, and SCIP. The solver backend determines optimization strength, while CTHR provides the source-consistent modeling layer and active-cell interface.
- **Audit certificate and provenance traceability.** For every returned decision, CTHR records the satisfied cell, instantiated constraints, active valid rule chain, and source provenance. The certificate is generated from the feasible cell and rule chain, not from a solver-specific proof log or a post-hoc natural-language explanation.
- **Two-domain 200-task benchmark.** This paper constructs and evaluates a systematic benchmark covering 150 aviation flight-procedure design tasks and 50 architecture/building-code design tasks. The benchmark separates method-visible inputs from hidden evaluation labels and supports evaluation of valid-rule identification, source-semantic decision validity, solver-backend reuse, rule-library quality, and audit traceability.

2. Related Work

2.1. Knowledge-Based Decision Support and Provenance

Knowledge-based decision support in regulated domains has long treated rules as executable objects rather than only retrievable text. Logic-program encodings of legislation, automated rule-based building-design checking, and provenance standards show that compliance systems must preserve both source meaning and audit evidence [6, 7, 8]. Recent KBS and neurosymbolic systems further combine symbolic structure with learned components for source-grounded reasoning [9, 10]. LLM/KG and KG/RAG studies show the value of explicit relational knowledge for organization, retrieval, graph completion, and explanation [11, 12, 13, 14, 15]. Similar source-grounded pipelines have appeared in legal, automated-driving, construction, manufacturing, and aviation workflows [16, 17, 18, 19, 20]. These systems strengthen source-grounded access and explanation, but they usually retrieve, validate, rank, or explain knowledge rather than compile provenance-rich rules into auditable optimization feasible regions. CTHR targets this missing modeling layer.

2.2. KG-to-Rule and LLM-Assisted Rule Modeling

KGs and declarative rule or constraint languages such as OWL/SWRL and SHACL provide natural representations for structured requirements, graph validation, and

rule-like integrity constraints [1, 2, 21]. LLM-assisted KG processing further reduces the knowledge acquisition burden by extracting candidate rules, relations, and provenance fields from technical corpora, as shown in KG-enhanced prompting, knowledge-based answer generation, hybrid RAG, and domain KG construction [22, 23, 24]. However, graph validation and rule extraction do not by themselves produce an optimization model: applicability, dependency, exclusion, override, precedence, and scenario parameters still determine which rules form a valid feasible region. CTHR therefore treats LLM/KG extraction as the front end and performs deterministic symbolic compilation before optimization.

2.3. Defeasible Reasoning and Symbolic Verification

Defeasible and non-monotonic reasoning are especially relevant to regulatory domains, where exceptions, priorities, and contextual specificity determine which rules survive [25, 26]. ASP and SMT solvers provide rigorous machinery for defaults, consistency checking, and formal verification [27, 5]. Neurosymbolic and KBS methods also use rules or graph structure as regularizers, differentiable logic, or relational representations for learning systems [28, 29, 30, 31]. These methods are powerful reasoning or verification tools, but they do not by themselves specify how source-linked KG evidence should be grounded, filtered, resolved, and translated into reusable continuous feasible cells. CTHR fills this role as a provenance-preserving compiler rather than a soft rule regularizer or representation learner.

2.4. Constrained and Multi-Objective Optimization

Constrained neural optimization and differentiable optimization layers improve feasibility or projection when a constraint set is already given [3, 32, 33, 34]. Preference-conditioned and evolutionary methods search for Pareto-efficient decisions under specified trade-offs [4, 35, 36, 37], and recent LLM-assisted planning or optimization work similarly combines generated procedural knowledge with formal execution or constrained search [38, 39]. These methods are downstream optimizers: they assume that the feasible region is already semantically correct. CTHR addresses the upstream KBS problem by compiling KG-induced rule semantics into the feasible region before any solver optimizes over it.

Taken together, prior work covers important fragments of the problem. Defeasible logics formalize priority and override; policy and compliance languages reason about permissions, prohibitions, obligations, and violations; planning, configuration, and constraint-generation methods instantiate rules into executable models; and ASP/SMT encodings can provide semantic guarantees for their target programs. CTHR targets a different compilation boundary. It starts from source-linked KG rule records whose active subset is scenario-dependent, preserves mutually exclusive and alternative compliance paths as separate valid rule structures, emits optimization-ready feasible cells over design variables, and attaches a certificate linking each optimized decision to rule and document provenance.

3. Motivation

In many high-assurance industrial settings, engineers do not merely face the problem of whether domain knowledge is available; they face the problem of how that knowledge can be operationalized. Flight procedure design and building code design provide representative examples. Domain specifications can be organized as KGs, where rules, clauses, conditions, and source links can be stored and retrieved. However, when an engineer needs to solve a concrete design problem, the KG usually only indicates which rules may be relevant. It does not directly tell an optimizer which variables may take which values, which rules are active in the current scenario, which rules should be excluded because of exceptions, mutual exclusions, or precedence, or what the resulting feasible region should be. In current practice, this transformation is often performed manually by a small number of experts who understand both the domain specification and optimization modeling. The motivation of this paper is to reduce this gap: KG rules should not remain only queryable knowledge, but should be transformable into executable, auditable, and optimizable feasible regions for concrete design scenarios.

The rapid progress of LLMs creates new opportunities for industrial knowledge modeling. LLMs can help read technical specifications, interpret domain terminology, summarize rule structure, and integrate knowledge across sources, making it more realistic to extract structured rule libraries from KG evidence. However, in safety-critical domains such as aviation and building design, final decisions should not be produced directly by probabilistic generation. They should be produced by deterministic, verifiable, and auditable mechanisms. A more appropriate division of labor is to use LLMs for the tasks at which they are effective, such as understanding text, organizing knowledge, and generating candidate rules, while assigning final rule resolution, constraint compilation, feasibility validation, and certificate generation to a formal symbolic framework. This view is consistent with LLM-Modulo planning [40], which argues that LLMs can help planning by providing knowledge and candidate structures, but planning, verification, and execution should remain grounded in external formal modules. CTHR follows the same principle for KG-driven optimization: LLMs and KGs assist rule-library construction, while a traceable symbolic rule compiler guarantees the verifiability and audibility of the final decision.

4. Problem Formulation

To formalize the semantic gap between symbolic knowledge and continuous geometric boundaries, a structured regulatory rule base is extracted from the source KG. This computational rule base encapsulates source-linked rule records, document-level provenance, and six explicit non-monotonic rule-interaction mechanisms: applicability, dependency, mutual exclusion, exception override, cross-source precedence, and scenario-dependent parameter propagation. Structurally, this rule base serves as the rigid deterministic

input for subsequent defeasible reasoning and polyhedral constraint compilation.

Definition 1 (Regulatory rule base). A regulatory rule base $\mathcal{R}$ induced by a source KG over a multi-dimensional scenario space Ω and a continuous decision space $\mathbb{R}^n$ consists of a finite set of source-linked regulatory assertions. Each rule $r \in \mathcal{R}$ is mapped to a parameterized consequent constraint $c_r(x, \eta(\omega)) \equiv \langle a_r(\eta(\omega)), x \rangle \leq b_r(\eta(\omega))$ and enriched with provenance metadata. Rules may be partitioned into disjoint conflict classes when their respective consequents represent alternative or mutually inconsistent compliance paths. The six core rule-interaction mechanisms are formally operationalized as follows:

- (i) An applicability function $\phi_r : \Omega \rightarrow \{0, 1\}$ defined for each rule, where $\phi_r(\omega) = 1$ dictates that rule r is triggered within the grounded scenario ω ;
- (ii) A directed dependency relation $\text{dep} \subseteq \mathcal{R} \times \mathcal{R}$, where $(r_1, r_2) \in \text{dep}$ denotes that the activation of rule r_1 logically requires the concurrent fulfillment of rule r_2 ;
- (iii) A symmetric exclusion relation $\text{excl} \subseteq \mathcal{R} \times \mathcal{R}$, where $(r_1, r_2) \in \text{excl}$ stipulates that r_1 and r_2 cannot simultaneously coexist in any valid deployment;
- (iv) An exception-override relation $\text{ovr} \subseteq \mathcal{R} \times \mathcal{R}$, where $(r_e, r_b) \in \text{ovr}$ indicates that a more specific contextual rule r_e defeats a general base rule r_b when both are triggered;
- (v) A scenario-dependent precedence strict partial order $>_\omega$ over conflicting rules within the same conflict class, induced by cross-document hierarchy or source specificity;
- (vi) A parameter propagation function $\eta : \Omega \rightarrow \mathbb{R}^p$ that maps high-dimensional scenario features to contextual parameter vectors. These parameters are not static empirical constants; instead, they are dynamically instantiated by the grounded scenario ω , thereby determining the precise numerical coefficients and affine boundaries utilized in downstream constraints.

Here, $\omega \in \Omega$ denotes a grounded domain scenario, and $x \in \mathbb{R}^n$ denotes a candidate decision vector within the engineering optimization domain. The coefficient vector $a_r(\eta(\omega)) \in \mathbb{R}^n$ and the scalar bound $b_r(\eta(\omega)) \in \mathbb{R}$ define the precise algebraic boundaries contributed by rule r after scenario-context grounding, where $\langle \cdot, \cdot \rangle$ denotes the standard Euclidean inner product. The integer p represents the dimension of the context-dependent parameter vector returned by η . Relational provenance is thoroughly indexed to preserve audit evidence, though it is excluded from the operational rule-interaction typologies.

Definition 2 (Defeasible resolution). For any grounded scenario $\omega \in \Omega$, let $\text{App}(\omega) = \{r \in \mathcal{R} : \phi_r(\omega) = 1\}$ denote the active candidate rule set. Within each designated conflict class, an applicable rule r_e is said to defeat r_b if and

only if $(r_e, r_b) \in \text{ovr}$ or $r_e >_\omega r_b$ holds. The valid rule set $\text{Valid}(\omega) \subseteq \text{App}(\omega)$ preserves all non-defeated applicable rules after the deterministic resolution of all override and precedence conflicts. Consequently, each maximal subset of $\text{Valid}(\omega)$ that remains pairwise consistent with excl and topologically closed under dep synthesizes a self-contained source-level extension, written as a valid rule structure chain(k). Let $\mathcal{K}_{\text{src}}(\omega) = \{1, \dots, K(\omega)\}$ be the ordered index set enumerating these alternative source-level extensions, satisfying $\text{chain}(k) \subseteq \text{Valid}(\omega)$ for every $k \in \mathcal{K}_{\text{src}}(\omega)$. This extension set is defined only from the source rule theory $(\mathcal{R}, \phi, \text{dep}, \text{excl}, \text{ovr}, >_\omega, \eta)$ and does not depend on any feasible cell emitted by CTHR.

In this formal resolution scheme, r_e characterizes the overriding or context-specific rule, r_b represents the defeated base rule, and $>_\omega$ tracks the dynamic priority relation mandated by cross-document authority boundaries. The integer $K(\omega)$ denotes the total number of alternative, self-consistent compliance structures yielded under scenario ω .

Definition 3 (Source rule-theory feasible semantics). The source rule-theory feasible region $F_{\text{src}}(\omega)$ for a designated scenario ω is defined directly from the source-level extensions of the rule theory:

$$F_{\text{src}}(\omega) = \left\{ x \in \mathbb{R}^n \mid \exists k \in \mathcal{K}_{\text{src}}(\omega) : \forall r \in \text{chain}(k), c_r(x, \eta(\omega)) \text{ holds} \right\}. \quad (1)$$

When emphasizing its KG origin, this independently defined source semantics is also written as $F_{\text{KG}}(\omega)$.

As formulated in (1), the global optimization space is represented not as a naive flat conjunction of all applicable requirements, but as a disjunctive semantic structure. A candidate decision vector x is deemed semantically valid if it satisfies the complete conjunction of constraints within at least one isolated valid rule structure chain(k). This formulation ensures that alternative compliance templates or mutually exclusive regulatory pathways are preserved as distinct geometric cells, preventing the spurious over-constraint or invalid optimization paths common in flat compilation baselines. Figure 1 illustrates this distinction using a building-code example with alternative roll-in shower templates.

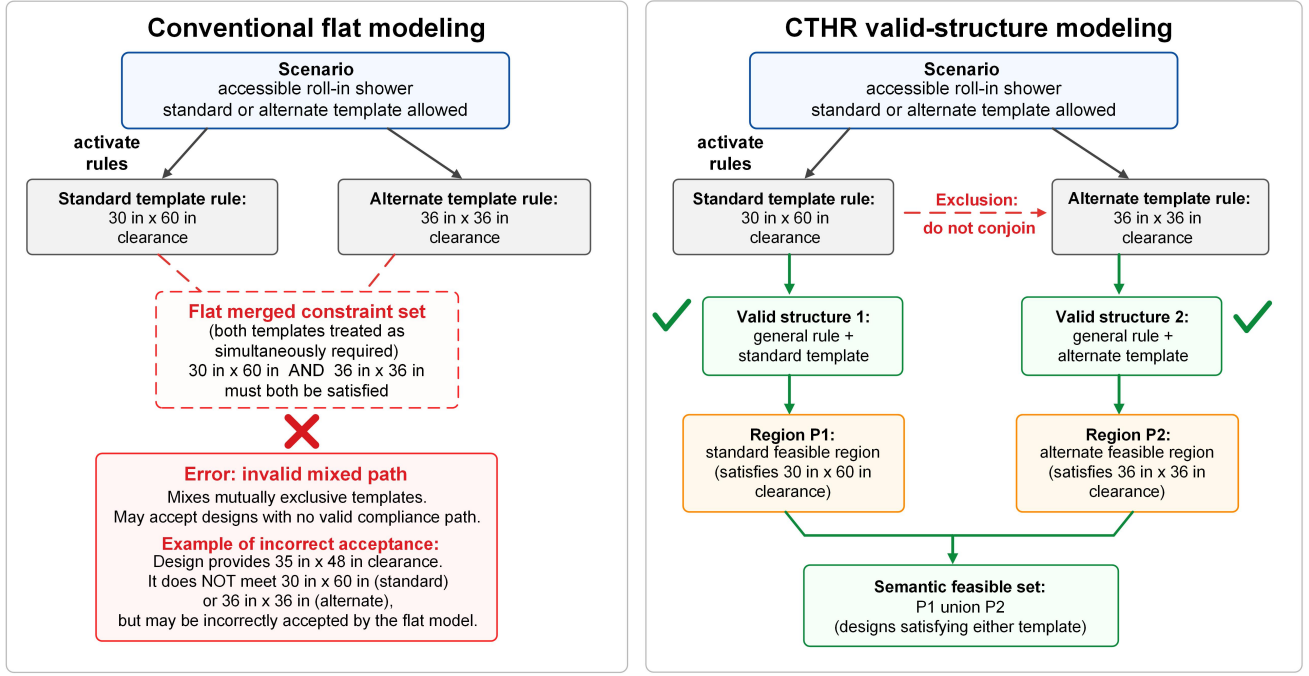


Figure 1: Illustrative comparison between conventional flat modeling and CTHR valid-structure modeling. Flat modeling may merge mutually exclusive compliance templates into a single constraint set, whereas CTHR preserves alternative valid rule structures and compiles them into a disjunctive semantic feasible region.

For a concrete engineering design query, the task provides a grounded scenario, decision variables, and objectives, while CTHR supplies the source-preserving feasible region $F_{KG}(\omega)$ and its provenance metadata. The downstream optimizer then searches within this emitted region, and the returned decision is evaluated against both the constructed feasible region and the hidden source-rule semantics in Section 7. Thus, the problem formulation isolates the contribution of CTHR to semantic feasible-region construction rather than to a particular objective scalarization or solver backend.

5. CTHR: Methodological Framework

The CTHR framework establishes a provenance-preserving knowledge-engineering pipeline that turns source-linked technical rules into problem-specific, auditable feasible cells for optimization. As shown in Figure 2, the CTHR method comprises four parts: Knowledge Acquisition, CTHR Modeling Layer, Solver Backends, and Optimized Decision & Audit Certificate.

The Knowledge Acquisition part constructs the knowledge graph and rule library. The CTHR Modeling Layer resolves task-valid rules and compiles them into source-traceable feasible cells. Solver backends then optimize the task objective within these cells; they are interchangeable execution engines rather than rule interpreters. Finally, the Optimized Decision & Audit Certificate component links each returned decision back to the active cell, valid rule chain, and source documents.

5.1. Knowledge Acquisition

Knowledge Acquisition converts unstructured technical specification documents into computable, traceable rule-based knowledge representations. The CTHR framework utilizes Cogniee to construct a source-linked knowledge graph, following the broader trend of using LLM-assisted knowledge engineering and KG/RAG pipelines to organize domain documents into structured, reusable knowledge assets [41, 10, 23, 19]. This process parses raw PDFs into granular text chunks while faithfully retaining source provenance metadata, including document names and page numbers, for each extracted fragment. Subsequently, Cogniee executes the structured extraction of regulatory entities, applicability criteria, numerical thresholds, measurement units, design objects, constraint relations, and cross-clause references from these text chunks, mapping them into a unified graph topology of nodes and edges. The resulting knowledge graph preserves both the literal normative content and the explicit provenance links connecting regulatory evidence to the original documents, thereby enabling downstream rules to be deterministically traced back to their specific source documents, clauses, or foundational evidence.

Following the construction of the knowledge graph, CTHR employs a Large Language Model (LLM) to transform the knowledge graph into a structured rule library. Operating as a KG-to-rule extractor, the LLM converts graph-structured evidence into uniformly formatted rule records, where each rule record encapsulates the original source text fragment,

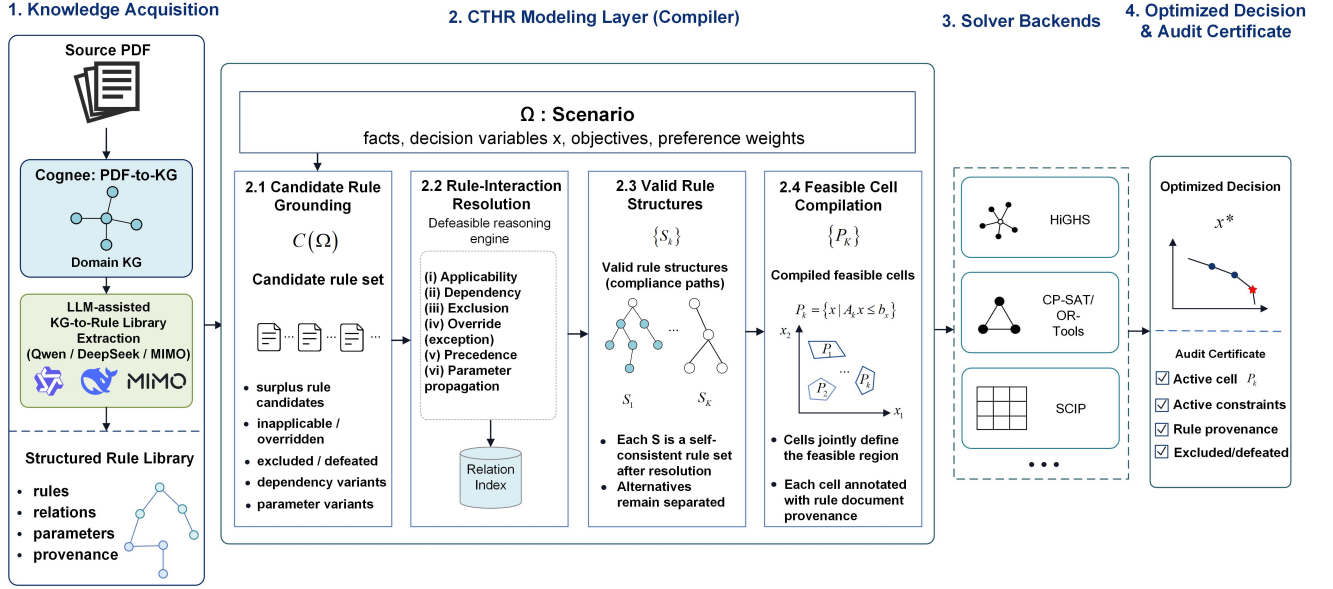


Figure 2: Overall automated end-to-end CTHR framework topology, mapping the pipeline from raw technical specifications to optimized decisions and audit certificates.

document identifier, section or page number prompts, associated entity nodes, numerical values, measurement units, and the local graph topology of adjacent relations.

The extraction prompt design for the LLM is structured around four core elements: first, task instructions that restrict the model to extracting technical specifications derived strictly from the provided graph evidence; second, input evidence comprising the source text chunk, related entities, numerical values, units, and local graph adjacency information; third, a target output schema specifying that every rule record must contain a rule identifier, rule name, domain, rule type, applicability guard, consequent constraints, parameters, inter-rule relations, and provenance fields; fourth, formatting constraints mandating a valid, parsable JSON structure while strictly prohibiting the fabrication of thresholds, units, or inter-rule relationships lacking explicit source evidence.

The generated rule records undergo automated verification and normalization to guarantee that their JSON format, rule identifiers, constraint expressions, measurement units, numerical sources, and provenance metadata fully satisfy subsequent symbolic modeling requirements. The validated rules are then compiled into a global rule library:

$$\mathcal{R} = \{r_i\}_{i=1}^N. \quad (2)$$

Each rule r_i encapsulates two distinct categories of information: the first is executable semantic information, such as applicability conditions, parameters, constraint expressions, and rule typologies; the second is structural and audit attributes, including dependency, exclusion, override, precedence, and parameter propagation relationships, supplemented by both document-level and rule-level provenance data.

5.2. CTHR Modeling Layer Candidate Rule Grounding

The objective of candidate rule grounding is to generate a high-recall candidate rule set $C(q)$ from the global rule library $\mathcal{R}$ for a specific optimization query q . Specifically, the framework computes a deterministic task-grounding score for each rule r_i in the rule library. This score comprehensively aggregates multiple matching metrics: the lexical token overlap between the optimization query text and the rule attributes, such as rule name, rule identifier, rule type, and the provenance section; the semantic alignment between fields and values within the applicability guards and the grounded scenario facts; the structural mapping of constraint variables to task decision variables; the exact consistency of measurement units; and the alignment of the originating document or source domain with the current task scope. For rules featuring explicit applicability guards, an additional weight is awarded if the guard conditions are satisfied within the current scenario. This scoring procedure is formulated as follows:

$$s(r_i, q) = s_{\text{text}} + s_{\text{guard}} + s_{\text{var}} + s_{\text{unit}} + s_{\text{prov}}. \quad (3)$$

Where s_{text} represents the lexical overlap component, computed by normalizing the visible task fields and rule meta-data into distinct token sets and evaluating their intersection size. This normalization pipeline involves lowercasing, tokenization via underscores, hyphens, and punctuation, removal of standard stopwords, and synonym expansion for domain-specific abbreviations. The remaining score components are derived from guard satisfaction, variable binding, unit consistency, and source document matching, respectively. Consequently, the task-grounding score operates as a deterministic, logic-driven symbolic matching metric rather than an embedding-based probabilistic vector similarity.

Subsequently, the framework ranks the rules based on their aggregated task-grounding scores and retains high-scoring records as initial candidates. Crucially, the output of this candidate grounding phase does not represent the final valid rule set, but rather an intentionally broad candidate set:

$$C(q) = \{r_i \in \mathcal{R} \mid s(r_i, q) \geq \tau \text{ or } r_i \text{ passes recall guard}\}. \quad (4)$$

This candidate set is deliberately over-inclusive and is typically significantly larger than the final resolved valid rule set.

Rule-Interaction Resolution

Candidate rule grounding favors recall, so $C(q)$ usually contains both active rules and rules recalled only through superficial matches in vocabulary, variables, or provenance. Some recalled rules also affect the final rule topology through dependency, exception, exclusion, precedence, or parameter propagation rather than through a direct numerical constraint. Rule-interaction resolution therefore contracts the broad candidate set into the valid rule set for the current task:

$$V(q) \subseteq C(q). \quad (5)$$

Algorithm 1 summarizes the procedure. It first applies task-profile filtering, then optionally invokes an LLM-assisted filter over public task and rule fields, and finally resolves explicit rule relations symbolically.

For task-profile filtering, CTHR builds

$$P(q) = \left(G_q^{\text{req}}, G_q^{\text{allow}}, G_q^{\text{block}}, \sigma_q \right), \quad (6)$$

where G_q^{req} , G_q^{allow} , and G_q^{block} are rule-group tags required, allowed, or blocked by the task, and σ_q enables strict filtering. Each candidate rule is mapped to group tags $G(r)$ using its identifier, name, type, source domain, guards, variables, and provenance. This mapping induces four operational states: *required* rules are explicitly triggered by the task; *allowed* rules remain compatible with the task domain or structure; *blocked* rules conflict with the scenario, design object, or domain; and *unmatched* rules have neither profile support nor structural support.

As shown in Algorithm 1, these states are converted into a profile score. Required and allowed matches receive positive weights, blocked and unmatched states receive penalties, and the visible binding term $v(r, q)$ captures guard-field overlap, bindable decision variables, unit consistency, and normalized token overlap. In the main experiments, the strict-profile configuration fixes $w_{\text{req}} = 7$, $w_{\text{allow}} = 3$, $w_{\text{block}} = 9$, $w_{\text{unmatch}} = 5$, and $\tau_{\text{seed}} = 8.0$; the last two weights enter the score with the minus signs shown in Algorithm 1. Rules that pass the seed threshold form $C_{\text{prof}}(q)$ after blocked/unmatched removal and required-rule re-injection. The optional LLM-assisted filter only uses public task descriptions, scenario facts, decision variables, candidate rule text, guards, and provenance fields; reference valid rules and solver outcomes are never exposed.

The final step performs symbolic resolution over $C_{\text{prof}}(q)$. CTHR removes rules whose guards are false, closes the set

Algorithm 1 Rule-Interaction Resolution

Require: Task q ; broad candidate set $C(q)$; rule relation graph $\mathcal{E}_{\text{rel}}$; optional LLM filter $\mathcal{M}$

Ensure: Valid rule set $V(q)$

- 1: Construct task profile $P(q) = (G_q^{\text{req}}, G_q^{\text{allow}}, G_q^{\text{block}}, \sigma_q)$
- 2: $C_{\text{seed}}(q) \leftarrow \emptyset$
- 3: **for** each $r \in C(q)$ **do**
- 4: Map rule r to rule groups $G(r)$
- 5: Compute profile indicators
 $I_{\text{req}}, I_{\text{allow}}, I_{\text{block}}, I_{\text{unmatch}}$
- 6: Compute visible binding score $v(r, q)$ from guard
fields, decision variables, units, and normalized tokens
- 7: Compute profile score: $s_{\text{prof}}(r, q) = w_{\text{req}}I_{\text{req}} +$
 $w_{\text{allow}}I_{\text{allow}} - w_{\text{block}}I_{\text{block}} - w_{\text{unmatch}}I_{\text{unmatch}} + v(r, q)$
- 8: **if** $s_{\text{prof}}(r, q) \geq \tau_{\text{seed}}$ **then**
- 9: $C_{\text{seed}}(q) \leftarrow C_{\text{seed}}(q) \cup \{r\}$
- 10: **end if**
- 11: **end for**
- 12: Remove blocked or unmatched rules from $C_{\text{seed}}(q)$
- 13: Re-inject required rules that are not blocked
- 14: Obtain $C_{\text{prof}}(q)$
- 15: **if** LLM-assisted filtering is enabled **then**
- 16: $C_{\text{prof}}(q) \leftarrow \mathcal{M}(q, C_{\text{prof}}(q))$
- 17: **end if**
- 18: $V(q) \leftarrow C_{\text{prof}}(q)$
- 19: Remove rules whose applicability guards are false under
 q
- 20: Close $V(q)$ under dependency and parameter/formula
propagation
- 21: Resolve exclusion, override, and precedence conflicts in
 $V(q)$
- 22: **return** $V(q)$

under dependency and parameter/formula propagation, and resolves exclusion, override, and precedence conflicts. The output is

$$V(q) = \text{Resolve} \left(C_{\text{prof}}(q), \mathcal{E}_{\text{rel}}, \phi_q \right), \quad (7)$$

where $\mathcal{E}_{\text{rel}}$ is the rule-relation graph and ϕ_q contains the task scenario facts, bindings, and unit profile.

Valid Rule Structure Construction

Rule-interaction resolution returns a valid rule set $V(q)$, but this set is still flat. Technical specifications may contain exceptions, parameter definitions, formulas, and alternative compliance paths that should not be merged into one conjunction. CTHR therefore organizes $V(q)$ into self-consistent valid rule structures:

$$\mathcal{S}(q) = \{S_1, S_2, \dots, S_{K(q)}\}, \quad (8)$$

where each S_k is independently compiled into one feasible cell.

CTHR first constructs the induced relation graph

$$H_q = (V(q), \mathcal{E}_{\text{rel}}|_{V(q)}), \quad (9)$$

where $\mathcal{E}_{\text{rel}}|_{V(q)}$ preserves the dependencies, exclusions, overrides, precedence orders, and parameter-propagation links among valid rules. Rules are then separated into core rules that must be retained and branch rules governed by alternative, exclusion, override, or precedence relations. Conflict classes are denoted as

$$\mathcal{B}(q) = \{B_1, B_2, \dots, B_m\}, \quad (10)$$

where each B_j contains rules or rule combinations that cannot co-exist. Override and precedence remove defeated branches; unresolved but mutually valid alternatives are retained as separate structures.

For each candidate structure, CTHR enforces dependency closure and parameter propagation:

$$\forall r \in S_k, \quad \text{Dep}(r) \cap V(q) \subseteq S_k. \quad (11)$$

Scenario-dependent parameters are instantiated from the grounded task facts. Structures with unsatisfiable guards, missing dependencies, or mutually exclusive rules are discarded.

The resulting valid rule structures are formally represented as:

$$S_k = (R_k, E_k, \gamma_k, \Pi_k), \quad (12)$$

where R_k is the rule subset, E_k is the induced relation sub-graph, γ_k is the activation condition, and Π_k stores rule- and document-level provenance. Each S_k must be dependency-closed and exclusion-consistent. Downstream compilation then processes each S_k independently, preserving the disjunctive semantics of alternative regulatory pathways.

Feasible-Region Compilation

Valid rule structure construction yields $S(q) = \{S_1, \dots, S_{K(q)}\}$, but each symbolic structure must still be converted into executable constraints. CTHR compiles every S_k into a feasible cell and aggregates these cells into the task-level search space.

For each rule r , the library stores compiled rule-to-constraint templates $\mathcal{T}(r) = \{t_{r,1}, t_{r,2}, \dots\}$. A template contains the constraint expression, required symbols, parameters, units, constraint role, and provenance mapping. At runtime, CTHR binds these symbols to the decision variables, bounds, unit system, and scenario parameters of task q .

For $S_k = (R_k, E_k, \gamma_k, \Pi_k)$, the compiler verifies required symbols, performs variable/unit/parameter binding, and merges the instantiated constraints with the baseline task domains:

$$P_k(q) = \left\{ x \in \mathcal{X}_q \mid \bigwedge_{r \in R_k} \bigwedge_{c \in \Gamma(r,q)} c(x; \eta_q) \text{ holds} \right\}, \quad (13)$$

where $\mathcal{X}_q$ is the baseline task domain, $\Gamma(r, q)$ is the executable constraint set emitted from rule r , and η_q contains propagated parameter values.

Alternative structures are kept disjunctive rather than flattened:

$$F_{\text{CTHR}}(q) = \bigcup_{k=1}^{K(q)} P_k(q). \quad (14)$$

Thus, a decision is compliant if it satisfies one complete feasible cell, which preserves alternatives and exceptions that flat conjunction would merge incorrectly.

Each cell also stores audit metadata:

$$\text{cell}_k = (\text{cell_id}, R_k, \Gamma_k, \gamma_k, \Pi_k, B_k), \quad (15)$$

where Γ_k contains instantiated constraints and B_k records bindings and parameter instantiations. These records allow a later solution to be mapped back to the active cell, valid rule structure, constraint templates, and source evidence. Overall, feasible-region compilation performs

$$S(q) \longrightarrow \{P_1(q), P_2(q), \dots, P_{K(q)}(q)\} \longrightarrow F_{\text{CTHR}}(q). \quad (16)$$

The compiled representation remains solver-agnostic.

5.3. Solver Backends

Following the processing within the CTHR modeling layer, the system outputs a set of feasible cells rather than a single mathematical programming model tied to one backend. CTHR is responsible for rule filtering, relational dependency resolution, valid rule structure construction, and feasible-cell compilation; the downstream backend is responsible for optimizing within the emitted search space. For a designated task q , the global feasible region synthesized by CTHR is the feasible-cell union in (14), where each $P_k(q)$ denotes a distinct feasible cell derived from a valid rule structure. The objective of the downstream solver backend is to search for an optimal decision vector within this search space:

$$\hat{x}_q \in \arg \min_{x \in F_{\text{CTHR}}(q)} g_q(x), \quad (17)$$

where $g_q(x)$ denotes the target objective function or a scalarized multi-objective formulation corresponding to the current task profile.

Crucially, the solver backends do not dynamically re-evaluate rule validity, nor do they re-resolve inter-rule dependencies, mutual exclusions, exception overrides, or precedence hierarchies. These semantic operations are handled upstream within the CTHR modeling layer. The downstream solver ingests finalized feasible cells, variable bounds, structural constraint expressions, and objective metrics. Consequently, the backends function as optimization engines rather than rule-interpretation modules.

Nevertheless, different solver backends require varied symbolic representations for the identical set of geometric feasible cells. For the ASP/clingo backend, CTHR translates the rule topology and cell membership rules into logical facts, choice rules, and integrity constraints. For the CP-SAT/OR-Tools backend, constraints are systematically transformed into integer-scaled linear equations, boolean decision variables,

and indicator conditions. For the SCIP backend, constraints are mapped into continuous, integer, linear, non-linear, or mixed-integer programming formulations depending on their structural expression types. Thus, while CTHR outputs a unified feasible region semantics, specialized backend adapters are deployed to compile the identical set of cells into the respective target modeling languages. Solver agnosticism is therefore an interface-level claim: CTHR does not automatically make every solver understand the emitted cells. A backend adapter must encode the cell-selection conditions, variables, constraints, bounds, and scalarized objective in the target solver language; solvers whose modeling language cannot express the required constraint class need an additional relaxation, reformulation, or verification layer.

This architectural design makes CTHR solver-agnostic. Default numerical optimizers, ASP/clingo, CP-SAT/OR-Tools, and SCIP can operate over the same CTHR feasible cells, while differing in their symbolic encodings, search mechanisms, and optimization strength. The output synthesized by a backend contains the optimized decision vector, objective values, and the active cell that the decision satisfies. CTHR then maps this active cell back to the corresponding valid rule structure, originating rules, constraint templates, and document-level source provenance.

In summary, CTHR defines the auditable feasible cells and their provenance links, while the selected backend determines how strongly the objective is optimized within those cells. This decoupling lets strong external solvers be used without requiring them to natively understand KG provenance or defeasible rule interactions, but each solver family still requires an explicit adapter for its own modeling language.

5.4. Optimized Decision and Audit Certificate

After solver execution, CTHR returns both the optimized decision and an audit certificate. The output is

$$o_q = (\hat{x}_q, g_q(\hat{x}_q), \text{cell}_k, \text{cert}_q), \quad (18)$$

where $\hat{x}_q$ is the decision vector, $g_q(\hat{x}_q)$ is its objective value, cell_k is the satisfied feasible cell, and cert_q is the certificate.

CTHR first identifies the active cell:

$$\hat{x}_q \in P_k(q). \quad (19)$$

Because each cell already stores $R_k, \Gamma_k, \mathcal{B}_k, \gamma_k$, and Π_k , the solution can be traced through

$$\hat{x}_q \rightarrow P_k(q) \rightarrow S_k \rightarrow R_k \rightarrow \text{Provenance}(R_k). \quad (20)$$

The certificate records four fields: the decision summary, the satisfied instantiated constraints, the active valid rule chain, and the source provenance. Since provenance links are preserved from document chunks to rule records and then to constraint templates, certificate generation is deterministic. It is not a solver proof log and not a post-hoc LLM explanation: the certificate is assembled from the selected feasible cell and its stored rule chain. If a constraint $c \in \Gamma_k$ is derived from rule r with provenance

$\Pi(r) = (\text{doc}, \text{page}, \text{section}, \text{chunk}, \text{kg_nodes})$, then the certificate records $c \rightarrow r \rightarrow \Pi(r)$:

$$\text{cert}_q = \{(c, r, \Pi(r)) \mid c \in \Gamma_k, r \in R_k\}. \quad (21)$$

This gives domain experts a direct path from a numerical decision to the executable constraints, valid rules, and original technical evidence that justify it.

6. Theoretical Analysis

The purpose of this section is to make explicit what must be guaranteed before the compiled knowledge graph (KG) can be safely used for optimization. Empirical results can show that CTHR works on the tested scenarios, but they do not by themselves prove that compilation preserves the constraint scope induced by the source KG. If a compiler retains defeated rules, merges mutually exclusive compliance paths, or ignores precedence, the optimizer may solve a feasible region that is either too large or too small. The theorem below compares CTHR against the independent source rule-theory semantics $F_{\text{src}}(\omega)$ in (1), which is defined before any feasible cell is emitted. This separation avoids using the compiled representation to define the semantic target. To formally bridge this theoretical analysis with the algorithmic framework detailed in Section 5, a grounded scenario $\omega \in \Omega$ is defined as the formal evaluation context mapped directly from the active runtime task query q and its associated scenario facts ϕ_q .

Theorem 1 is the main correctness result for this compilation target. It is not claimed as a general semantic-preservation theorem for all symbolic encodings; rather, it states the property needed by CTHR: the emitted feasible-cell union is sound and complete with respect to the independently defined source rule theory. A proof sketch is given below, and the detailed lemma proofs are provided in Appendix B.

Assumption 1 (Well-founded defeat). *The defeat relation over conflict classes is acyclic and uniquely determined by scenario ω (no cycles in override/precedence chains after normalization).*

Assumption 2 (Acyclic dependencies). *The dep relation is acyclic. Parameter propagation $\eta(\omega)$ is uniquely determined from ω by topological evaluation.*

Assumption 3 (Polytope non-emptiness). *Each feasible cell P_k is a nonempty, closed polytope within the bounded design space.*

In the practical pipeline, the geometric property required by Assumption 3 is rigorously enforced during the compilation layer. CTHR preemptively invokes the downstream constraint programming backend (specifically, Google OR-Tools CP-SAT) to execute a structural feasibility check, filtering out inherently self-contradictory or unresolvable rule structures before they are emitted to final optimization.

Theorem 1 (Source-Theory Soundness and Completeness). *Under Assumptions 1–3, let the emitted CTHR feasible region for scenario ω be*

$$F_{\text{emit}}(\omega) = \bigcup_{\substack{k \in \mathcal{K}_{\text{src}}(\omega) \\ \omega \models \gamma_k}} P_k. \quad (22)$$

For every grounded scenario $\omega \in \Omega$ and decision $x \in \mathbb{R}^n$, the compiler is:

$$\text{sound: } x \in F_{\text{emit}}(\omega) \Rightarrow x \in F_{\text{src}}(\omega), \quad (23)$$

$$\text{complete: } x \in F_{\text{src}}(\omega) \Rightarrow x \in F_{\text{emit}}(\omega). \quad (24)$$

Consequently, $F_{\text{emit}}(\omega) = F_{\text{src}}(\omega)$, and any downstream optimizer constrained to the emitted feasible cells solves over exactly the feasible decision scope induced by the source rule theory.

The proof decomposes into five lemmas:

Lemma 1 (Applicability Correctness, L1). *For every scenario ω and valid rule structure k : $\omega \models \gamma_k$ iff every rule in $\text{chain}(k)$ is applicable, the chain is closed under dependencies, and the chain is internally consistent with the exclusion relation. Applicable alternatives outside the chain may still exist and form separate source-level extensions.*

Lemma 2 (Override Correctness, L2). *Let r_e, r_b be applicable rules in the same conflict class with r_e overriding r_b . Then for every $k \in \mathcal{K}_{\text{src}}(\omega)$, the valid rule structure for k contains the consequent of r_e and excludes the defeated consequent of r_b .*

Lemma 3 (Precedence Correctness, L3). *Let $r_i \succ_{\omega} r_j$ be conflicting applicable rules. Every valid rule structure retains the consequent of r_i and excludes the conflicting consequent of r_j . Non-conflicting applicable rules are both retained.*

Lemma 4 (Parameter Propagation Correctness, L4). *For every propagated rule r , scenario ω , and design point x : $c_r(x, \eta(\omega)) \iff \hat{c}_{r,\omega}(x)$, where $\hat{c}_{r,\omega}$ is the compiler-instantiated constraint after symbolic substitution of $\eta(\omega)$.*

Lemma 5 (Geometry Correctness, L5). *For every feasible cell k with $\omega \models \gamma_k$: $x \in P_k \iff x$ satisfies all surviving instantiated consequents of $\text{chain}(k)$.*

Proof of Theorem 1 (sketch). Soundness: Let $x \in F_{\text{emit}}(\omega)$. Then $x \in P_k$ for some $k \in \mathcal{K}_{\text{src}}(\omega)$ with $\omega \models \gamma_k$. By L5, x satisfies all emitted instantiated consequents of $\text{chain}(k)$. By L2–L3, these consequents are exactly the non-defeated source-level consequents retained by the source rule theory. By L4, propagated parameter instantiation is source-equivalent. By L1, $\text{chain}(k)$ is an applicable, dependency-closed, exclusion-consistent source-level extension. Therefore x satisfies the definition of $F_{\text{src}}(\omega)$.

Completeness: Let $x \in F_{\text{src}}(\omega)$. By (1), there exists a source-level extension $\text{chain}(k)$ such that all source-level consequents in that extension hold at x . By L1, the compiler activation condition satisfies $\omega \models \gamma_k$. By L2–L3, the compiler

removes exactly the defeated alternatives and preserves the same non-defeated extension. By L4, parameter substitution preserves each source consequent. By L5, the instantiated consequents define P_k , so $x \in P_k \subseteq F_{\text{emit}}(\omega)$. $\square$

Remark 1. *Theorem 1 is a conditional soundness-and-completeness result relative to the independently defined source rule-theory semantics in (1). It does not state that the entire end-to-end pipeline always recovers the correct source-level extension. Therefore, invalid cases observed in Section 7.2 should be interpreted as pipeline errors from candidate-to-valid recovery, rule-library quality, or backend optimization over the emitted feasible cells, rather than as a contradiction of the theorem itself.*

7. Experiments

The experiments test the central claim of this paper: a source KG must be compiled through valid rule structures before its rules are used as optimization constraints. Section 7.2 evaluates end-to-end KG-to-constraint modeling and solver-backend reuse. Section 7.3 then isolates the candidate-to-valid rule recovery step before feasible-region compilation and optimization. Section 7.4 evaluates how different LLM-generated rule libraries affect the same CTHR modeling pipeline. Section 7.5 finally evaluates whether the optimized decisions returned through different solver backends retain complete audit certificates. Section 7.6 studies which CTHR components are responsible for the observed end-to-end performance, and Section 7.7 reports supplemental fairness and controlled scalability checks.

7.1. Experimental Setup

Domain Corpora and KG Exports

The experiments use two source-traceable domain corpora. The aviation corpus is derived from the Civil Aviation Administration of China (CAAC) flight-procedure design specification AC-97-FS-005R1. The architecture corpus combines three regulatory sources: the 2010 ADA Standards for Accessible Design, the 2021 International Building Code (IBC), and the 2021 International Fire Code (IFC). Each corpus is converted into a source-linked knowledge graph (KG) using Cogneer [41]. The exported data fully retains chunk identifiers, entity nodes, KG edges, document labels. Table 1 summarizes the specific statistics of the two KG exports used in the experiments.

Benchmark Tasks

A controlled source-grounded benchmark is constructed with 200 rule-grounded design and compliance tasks: 150 aviation flight-procedure design tasks and 50 architecture/building-code design tasks. Task scenarios are curated from source specifications and domain practice scenarios; they are not generated from the parsed KG, rule library, feasible cells, or solver results. The aviation tasks are derived from CAAC AC-97-FS-005R1. To audit aviation reference-label quality, 100 sampled aviation tasks were reviewed by three flight-procedure design experts

Table 1

Domain corpora and KG exports used for the benchmark.

Domain	Source corpus	KG nodes	KG edges
Aviation	CAAC flight-procedure design specification AC-97-FS-005R1 (refers to ICAO Doc 8168).	4,659	6,876
Architecture	2010 ADA Standards for Accessible Design; 2021 International Building Code; 2021 International Fire Code.	42,350	61,239

certified through the CAAC Flight Procedure Design Basic Knowledge Examination; Appendix A reports the review protocol and scores. The architecture tasks are grounded in the ADA, IBC, and IFC rule sources and adapt scenario themes from NCARB ARE practice materials after rewriting, parameterization, and mapping to the architecture rule library. The architecture benchmark does not reproduce original question stems, answer options, or official explanations, and was manually reviewed.

Each task stores algorithm-visible inputs, a public scenario model, and evaluator-only references. Visible inputs include task metadata, scenario facts, decision variables, objectives or scalarized preferences, and the public scenario model; hidden references contain surviving rule IDs, executable rule-derived constraints, and certificate targets. The aviation expert audit reports aggregate quality scores rather than formal inter-annotator agreement. Figure 3 shows a visible task input from the two domains.

Method-Visible Inputs and Hidden Evaluation Labels

To avoid oracle leakage, every experiment separates method inputs from evaluation labels. The method-visible inputs include the task domain, task type, scenario facts, public scenario model, decision variables and units, objectives or preference weights, the generated rule library, candidate rules available to the evaluated method, and the rule relations recorded in that library. Hidden evaluation labels include reference valid rule IDs, reference valid rule structures, rule-derived semantic constraints, certificate targets, invalid-case labels, and best-known objectives used for objective-gap reporting. These hidden fields are read only by the evaluator when computing Rule P/R, Sem-CSR, invalid cases, certificate completeness, and objective gaps. They are not supplied to candidate grounding, the profile resolver, the optional LLM filter, feasible cell compilation, or the solver backends. Experiments that intentionally fix feasible cells or use oracle-aligned candidates are marked as such and are used only to isolate solver reuse or rule-library usability from earlier grounding failures.

Concretely, method-visible files are the core algorithm-input files, public scenario-model files, and the selected rule libraries. Evaluator-only files include source-semantic references, evaluation references, evaluator overlays, compiled evaluator templates, and the evaluation_reference portion of paired task files.

Paired task files containing both sides are retained only as audit artifacts: experimental runners use separated algorithm-input, scenario-model, and evaluation-reference files, and references are loaded only after prediction. Forbidden-key leakage checks are run on the separated files before evaluation.

The 200 tasks collectively exercise the main rule-interaction mechanisms used by CTHR, but coverage is intentionally uneven across domains and should be read as a curated source-semantic audit rather than a balanced sampling distribution. The architecture tasks emphasize rule applicability, multi-rule conjunction, exception handling, precedence, and low-dimensional regulatory thresholds. The aviation tasks include both business-realistic multi-constraint procedure-design queries and rule-grounded cases targeting applicability, dependency, branch exclusion, precedence, parameter propagation, and audit traceability.

KG-to-Rule-Library Extraction Models

Starting from the fixed KG exports, three LLM backbones are called to generate rule libraries: qwen-plus, deepseek-v4-pro, and mimo-v2.5-pro. Aviation and architecture use domain-specific KG-to-rule prompt templates, which are archived with the supplementary experiment scripts. Both prompts use the same output schema: each returned rule must contain a stable rule identifier, human-readable name, domain, rule type, source chunk IDs, optional KG node IDs, a guard, constraints, inter-rule relations, provenance fields, extraction notes, and a confidence score. The model is instructed to return only grounded JSON, to avoid inventing numeric values or source references, and to preserve exceptions, precedence, alternatives, and dependencies as explicit rule relations.

Metrics

CTHR and the compared methods are evaluated from five perspectives: valid-rule identification, feasible-region and decision validity, solver-backend performance, rule-library quality, and audit traceability. Unless otherwise specified, ratio-based metrics are first computed for each task and then averaged over the corresponding dataset.

Valid-rule identification metrics. To evaluate whether a method can identify the rules that are truly valid for a task, rule precision and rule recall are reported. For a task q , let $R_{\text{pred}}(q)$ denote the predicted valid rule set and $R_{\text{ref}}(q)$ denote the reference valid rule set. These metrics are defined as

$$\text{Rule Precision}(q) = \frac{|R_{\text{pred}}(q) \cap R_{\text{ref}}(q)|}{|R_{\text{pred}}(q)|}, \quad (25)$$

$$\text{Rule Recall}(q) = \frac{|R_{\text{pred}}(q) \cap R_{\text{ref}}(q)|}{|R_{\text{ref}}(q)|}. \quad (26)$$

In the candidate-to-valid experiment, Candidate/Ref Ratio and Predicted/Ref Ratio are additionally reported. Candidate/Ref Ratio is the number of candidate rules divided by

Aviation: AVI_SMIX_019 Precision approach glide path with controlling obstacle Task metadata Domain: aviation_procedure_design Task focus: business_multi_constraint Task type: precision_approach_design Design intent Select a glide path angle while balancing descent rate, final approach length pressure, and obstacle protection margin. Scenario facts procedure_context: precision approach minimum_required_obstacle_clearance_m: 65 controlling_obstacle_id: OBS_019_1 obstacle_along_track_distance_nm: 3.8 obstacle_top_height_m: 218 Decision variables glide_path_angle_deg: continuous, degree, [2.0, 4.5] ground_speed_kt: continuous, kt, [80, 180] descent_rate_ft_min: continuous, ft/min, [250, 1200] control_obstacle_margin_m: continuous, m, [-300, 1200] Objectives minimize_descent_rate: descent_rate_ft_min maximize_obstacle_margin: control_obstacle_margin_m minimize_length_pressure: final_approach_length_pressure Public scenario model visibility: public_algorithm_input B1: descent_gradient_percent >= 1.745 * glide_path_angle_deg B2: descent_rate_ft_min >= 1.0127 * ground_speed_kt * descent_gradient_percent O1: procedure_design_altitude_m >= obstacle_top_height_m + required_obstacle_clearance_m	Architecture: ARCH_FKG_01 Means-of-egress ramp for clinic level change Task metadata Domain: architecture_code_compliance Source domain: IFC Task type: egress_ramp_design Design intent Choose ramp slope and landing dimensions for a clinic means-of-egress ramp, balancing compact run length and landing space. Scenario facts egress_component: ramp ramp_usage: means_of_egress change_in_direction: true rise_ft: 2 Decision variables running_slope: continuous, ratio, [0.02, 0.12] landing_minimum_dimension_in: continuous, inch, [36, 84] ramp_run_length_ft: continuous, ft, [10, 105] Objectives minimize_ramp_run_length: ramp_run_length_ft maximize_landing_dimension: landing_minimum_dimension_in Public scenario model visibility: public_algorithm_input C90: ramp_run_length_ft >= rise_ft / running_slope
--	--

Figure 3: Example algorithm-visible task inputs from the aviation and architecture benchmarks. The figure shows only the public task metadata, scenario facts, decision variables, objectives, and public scenario model. Hidden rule IDs, rule-derived feasible-region constraints, and provenance targets are used only for evaluation.

the number of reference valid rules, measuring how broad the grounding input is. Predicted/Ref Ratio is the number of predicted valid rules divided by the number of reference valid rules, measuring whether the recovered rule-set size is close to the reference. Exact Rule Match is also reported as the percentage of tasks for which the predicted rule set exactly matches the reference rule set.

Feasible-region and decision-validity metrics. To evaluate whether the optimized decisions satisfy the relevant constraint semantics, Formal CSR and Semantic CSR are reported. Formal CSR checks whether the returned decision satisfies the feasible region constructed by the evaluated method itself:

$$\text{Formal CSR} = \frac{|\{q \in Q : \hat{x}_q \in F_{\text{method}}(q)\}|}{|Q|}. \quad (27)$$

Semantic CSR checks whether the same returned decision satisfies the hidden source-rule reference semantics:

$$\text{Semantic CSR} = \frac{|\{q \in Q : \hat{x}_q \in F_{\text{ref}}(q)\}|}{|Q|}. \quad (28)$$

Formal CSR measures internal modeling and solver consistency, whereas Semantic CSR measures whether the returned decision is genuinely valid under the source-rule semantics. False Accept and Invalid Cases are also reported. False Accept is the percentage of tasks where the method accepts its returned decision as feasible but the reference semantics rejects it. Invalid Cases count the tasks whose final optimized decision violates the reference semantics.

Solver-backend metrics. To evaluate whether different solvers can consume the same fixed CTHR feasible cells, Solve, Cell CSR, and Objective Gap are reported. Solve is the percentage of tasks for which the backend returns an optimized decision:

$$\text{Solve} = \frac{|\{q \in Q : \hat{x}_q \text{ returned}\}|}{|Q|}. \quad (29)$$

Cell CSR checks whether the returned decision satisfies the CTHR feasible cell:

$$\text{Cell CSR} = \frac{|\{q \in Q : \hat{x}_q \in P_k(q)\}|}{|Q|}. \quad (30)$$

Objective Gap measures the relative gap between the objective value returned by a solver and the best-known objective for the same task, averaged over tasks. This metric compares optimization quality across solver backends over the same feasible cells, rather than rule-recovery quality.

Rule-library quality metrics. To compare rule libraries generated by different LLM backbones, both record-level and downstream metrics are reported. Rules is the total number of rule records generated by a given model. Provenance Valid checks whether each rule record can be traced to a valid source document, clause, or evidence chunk. In the downstream evaluation, Canonical Precision/Recall first maps model-specific rule IDs back to canonical source-rule IDs before computing rule precision and recall, avoiding false mismatches caused by different ID namespaces. Cand.-0 counts tasks for which the oracle-candidate input is empty, and Invalid counts optimized decisions that violate the source-rule semantic oracle.

Audit-traceability metrics. To evaluate auditability, Certificate Coverage, Rule Provenance Valid, and Valid-chain Trace Complete are reported. Certificate Coverage is the percentage of task-method rows for which an optimized decision is returned together with an audit certificate. Rule Provenance Valid checks whether the rule IDs listed in the certificate can be found in the rule library or KG provenance records. Valid-chain Trace Complete checks whether the certificate lists the complete valid rule structure corresponding to the active feasible cell. These metrics verify whether CTHR can trace an optimized decision back to the rule chain and original document evidence that support it.

7.2. Overall Performance Evaluation

This section evaluates CTHR in two settings. The first is the full KG-to-constraint pipeline, where each method starts from candidate rules, constructs its own feasible region, and returns an optimized decision. Flat and native symbolic baselines consume candidate rules without the CTHR valid-rule-structure layer. The CTHR-style rows use the same CTHR candidate-to-valid resolution and feasible-cell compilation, replacing only the backend encoding and optimizer. The current 200-task evaluation reports the numeric backends used for the new benchmark: HiGHS, CP-SAT/OR-Tools, and SCIP. The second setting fixes the feasible cells compiled by CTHR and changes only the downstream solver, isolating backend reuse.

Table 2 shows that the flat aviation baseline still retrieves many reference rules, with 91.7% recall, but it flattens incompatible and underspecified candidates into a single constraint set and produces no source-valid decisions. Native symbolic encodings are also weak on aviation because they must both select valid rules and encode a heterogeneous mix of numeric, geometric, and formula-derived constraints without the CTHR valid-structure layer. Their Sem-CSR remains between 8.7% and 10.7%.

CTHR improves the aviation result by separating rule selection from feasible-cell compilation. The CTHR-style

HiGHS and SCIP rows reach 88.7% Sem-CSR and reduce invalid tasks from 150/150 under the flat baseline to 17/150. The CP-SAT/OR-Tools backend is slightly lower at 86.0% Sem-CSR. These numbers should not be read as a claim that backend choice is irrelevant: candidate-to-valid recovery is imperfect on the 150-task aviation suite (74.7% precision and 70.7% recall), and the remaining false accepts indicate cases where the compiled cell or backend solution still fails the hidden source-rule semantics.

The architecture benchmark presents a different pattern. Flat modeling has very high recall (99.0%) but very low precision (6.8%), yielding 78.0% Sem-CSR and 11/50 invalid tasks. Native CP-SAT/OR-Tools and SCIP perform strongly on this lower-dimensional building-code suite, reaching 92.0% Sem-CSR, but they still show 8.0% false accept and four invalid tasks. The CTHR-style backends improve rule identification to 95.3% precision and 95.0% recall, remove false accepts, and reach 96.0% Sem-CSR with 2/50 invalid tasks. Thus, CTHR is not a new optimizer; its contribution is the source-aware modeling layer that turns KG-derived rules into feasible cells that can then be optimized by different backends.

Table 3 shows that, once CTHR feasible cells are fixed, backend behavior becomes much clearer. On architecture, all three reported backends solve all 50 tasks and satisfy the fixed cells. On aviation, HiGHS, CP-SAT/OR-Tools, and SCIP solve 96.7% of the 150 tasks; HiGHS and SCIP return only cell-satisfying solutions, while CP-SAT/OR-Tools has a slightly lower Cell CSR of 95.3%. Objective gaps are small for HiGHS and SCIP on aviation, while architecture shows that SCIP obtains the tightest objectives over the same cells. These results support the intended division of labor: CTHR defines the auditable feasible cells, and the solver backend determines how completely and tightly those cells are optimized.

7.3. Candidate-to-Valid Rule Recovery

The previous section evaluates the complete modeling-and-solving pipeline. This section isolates the rule-recovery step before any feasible-region compilation or downstream optimization is considered. The goal is to test whether CTHR can recover the valid rules for a grounded scenario from the candidate set. The broad candidate set is constructed using only method-visible task fields, public scenario facts, decision variables, rule metadata, guards, relation fields, provenance, and source-domain hints. Reference valid rules, hidden constraints, reference feasible cells, semantic validators, and solver outcomes are not exposed to candidate grounding, profile filtering, the optional LLM reranker, or the final relation-selection step; they are used only after prediction for evaluation. In this experiment, CTHR uses the current candidate-to-valid profile resolver. The aviation run uses the aviation recall guard without online LLM filtering, while the architecture run uses the cached profile-auto resolver.

A set of candidate-to-valid rule-identification metrics is used to evaluate whether CTHR can identify task-valid rules from candidate rules. Candidate/Ref is the size of the input broad candidate set $C(q)$ in Algorithm 1, normalized by the

Table 2

End-to-end KG-to-constraint modeling performance on the 200-task aviation and architecture benchmark. Each method starts from candidate rules, constructs a feasible region, and returns an optimized decision.

Dataset	Method	Type	Rule P	Rule R	Formal CSR	Sem-CSR	False accept	Invalid
Aviation	Flat baseline	Flat	7.9%	91.7%	0.0%	0.0%	0.0%	150/150
Aviation	Native MILP + HiGHS	Native symbolic	2.9%	7.3%	28.0%	8.7%	19.3%	137/150
Aviation	Native CP-SAT + OR-Tools	Native symbolic	3.3%	8.3%	31.3%	10.7%	20.7%	134/150
Aviation	Native SCIP	Native symbolic	3.3%	8.3%	31.3%	8.7%	22.7%	137/150
Aviation	CTHR-style HiGHS	CTHR cells	74.7%	70.7%	96.7%	88.7%	8.0%	17/150
Aviation	CTHR-style CP-SAT + OR-Tools	CTHR cells	74.7%	70.7%	94.0%	86.0%	8.0%	21/150
Aviation	CTHR-style SCIP	CTHR cells	74.7%	70.7%	96.7%	88.7%	8.0%	17/150
Architecture	Flat baseline	Flat	6.8%	99.0%	66.0%	78.0%	0.0%	11/50
Architecture	Native MILP + HiGHS	Native symbolic	51.7%	62.0%	72.0%	64.0%	8.0%	18/50
Architecture	Native CP-SAT + OR-Tools	Native symbolic	75.7%	90.0%	100.0%	92.0%	8.0%	4/50
Architecture	Native SCIP	Native symbolic	75.7%	90.0%	100.0%	92.0%	8.0%	4/50
Architecture	CTHR-style HiGHS	CTHR cells	95.3%	95.0%	96.0%	96.0%	0.0%	2/50
Architecture	CTHR-style CP-SAT + OR-Tools	CTHR cells	95.3%	95.0%	96.0%	96.0%	0.0%	2/50
Architecture	CTHR-style SCIP	CTHR cells	95.3%	95.0%	96.0%	96.0%	0.0%	2/50

Table 3

Solver-backend evaluation over fixed CTHR feasible cells. The inputs are the latest CTHR predicted valid rules and compiled constraint templates; only the downstream solver backend is changed.

Dataset	Solver over CTHR cells	Solve	Cell CSR	Objective gap
Aviation	HiGHS over CTHR cells	96.7%	96.7%	0.0%
Aviation	CP-SAT + OR-Tools over CTHR cells	96.7%	95.3%	0.7%
Aviation	SCIP over CTHR cells	96.7%	96.7%	0.0%
Architecture	HiGHS over CTHR cells	100.0%	100.0%	2.0%
Architecture	CP-SAT + OR-Tools over CTHR cells	100.0%	100.0%	2.0%
Architecture	SCIP over CTHR cells	100.0%	100.0%	0.0%

number of reference valid rules. Filtered/Ref is the size of the profile-constrained set $C_{\text{prof}}(q)$ obtained after lines 1–14, normalized by the same reference count. Predicted/Ref is the size of the final valid rule set $V(q)$ returned by Algorithm 1, again normalized by the reference count. Rule-ID precision and recall measure the set overlap between predicted and reference rule identifiers. Exact match is the percentage of tasks for which the predicted and reference rule sets are identical. Extra and Missing count non-reference rules included in the prediction and reference rules missed by the method, respectively.

Table 4 shows that the candidate sets are substantially broader than the reference valid-rule sets. Aviation starts from 11.237 times as many candidate rules as reference rules on average. CTHR reduces the intermediate Filtered/Ref ratio to 1.413 and the final Predicted/Ref ratio to 0.960, but the enlarged aviation benchmark remains challenging: Rule-ID precision is 74.7%, Rule-ID recall is 70.7%, and the latest exact-match audit is 70.0%.

Architecture begins from an even wider candidate pool, 15.387 times the reference size, but the profile-auto resolver contracts it to a Filtered/Ref ratio of 1.670 and a final Predicted/Ref ratio of 1.340. It maintains 93.3% Rule-ID precision, 93.0% Rule-ID recall, and 90.0% exact match. The architecture audit also reports 70.0% semantic constraint satisfaction and 92.0% formal feasibility for this isolated candidate-to-valid run, indicating that some failures after rule identification arise from downstream solving or semantic checking rather than from rule-ID recovery alone.

Table 4

Aggregate candidate-to-valid rule recovery results. Candidate/Ref measures the breadth of the method-visible grounded candidate set, Filtered/Ref measures the intermediate profile-constrained candidate set, and Predicted/Ref measures the final CTHR-selected valid-rule set, all normalized by the reference valid-rule count used only for evaluation.

Dataset	Tasks	Candidate / Ref	Filtered / Ref	Predicted / Ref	Rule-ID precision	Rule-ID recall	Exact match
Aviation	150	11.237	1.413	0.960	0.747	0.707	0.700
Architecture	50	15.387	1.670	1.340	0.933	0.930	0.900

Table 5

Quality audit of LLM-generated rule libraries. The Rules column counts all rule records emitted by the generator. Provenance validity checks whether each rule can be traced to the expected source document, clause, or evidence chunk.

Dataset	Generator	Rules	Provenance valid	Constraint grounding	Relation grounding
Aviation	qwen-plus	111	100.0%	100.0%	100.0%
Aviation	DeepSeek-v4-Pro	142	100.0%	99.1%	99.6%
Aviation	MIMO-V2.5-Pro	186	100.0%	100.0%	100.0%
Architecture	qwen-plus	1,585	100.0%	100.0%	100.0%
Architecture	DeepSeek-v4-Pro	2,419	100.0%	99.5%	99.6%
Architecture	MIMO-V2.5-Pro	2,157	99.9%	99.6%	99.8%

7.4. Rule-Library Generator Comparison

The previous experiments use the qwen-plus generated rule library as the default rule library for each domain. This section therefore treats qwen-plus as the main rule-library source and uses DeepSeek-v4-Pro and Xiaomi MIMO-V2.5-Pro as two additional replacement libraries to test whether the modeling layer can consume rules produced by different LLMs. The evaluation has two levels. First, the generated rule records themselves are audited in terms of rule count, provenance validity, constraint grounding, and relation grounding. Second, each model-specific rule library is substituted into a downstream CTHR run while keeping task parameters, source-rule semantic oracle, and solver backend fixed. This experiment is diagnostic: it tests rule-library usability under controlled alignment and compilation support, not the success rate of a fully automatic KG-to-rule extraction pipeline.

Table 5 shows that the three generators all produce largely well-grounded rule records. Provenance validity is at least 99.9% across all six domain-generator pairs, and constraint and relation grounding are also above 99.1%. The generators differ more in rule-library size and downstream usability than in whether the emitted records contain traceable source references.

Because different LLMs may express the same source rule with different granularities, rule names, and textual descriptions, direct evaluation with model-specific rule IDs can introduce alignment failures. The evaluation therefore maps model-specific rules to canonical source-rule identifiers before computing Rule P/R. Table 6 uses the same SCIP backend for all rows. The qwen-plus rows are the default rule-library results and therefore match the corresponding CTHR-style SCIP rows in Table 2. DeepSeek-Pro and Xiaomi MIMO are replacement libraries evaluated under the same backend and task set. In aviation, all three libraries avoid candidate-empty and valid-rule-empty failures. The default

qwen-plus reaches 88.7% Sem-CSR with 17/150 invalid tasks. DeepSeek-Pro achieves a higher rule precision of 76.8% and a comparable recall of 70.0%, yielding 86.7% Sem-CSR and 15/150 invalid cases. Xiaomi MIMO exhibits the highest rule precision at 78.9%, but its Sem-CSR drops to 84.0% with 19/150 invalid cases. In architecture, all three libraries similarly avoid candidate-empty and valid-rule-empty failures, uniformly reaching 96.0% Sem-CSR with exactly 2/50 invalid tasks, varying only slightly in their rule precision and recall granularities. These results indicate that CTHR can consume multiple generated rule libraries under controlled alignment, but downstream validity still depends on generator-specific rule granularity and executable semantics.

Table 6

Downstream performance when replacing the rule library generator under a controlled CTHR setting. The solver backend is fixed to SCIP, matching the CTHR-style SCIP rows in Table 2. Rule P/R, Formal CSR, Sem-CSR, and False accept are reported as percentages. Cand.-0 counts tasks with no candidate rules, and Valid-0 counts tasks for which no valid rules remain after CTHR recovery.

Dataset	Generator	Backend	Rule P/R	Formal CSR	Sem-CSR	False acc.	Cand.-0	Valid-0	Invalid
Aviation	qwen-plus	SCIP	74.7%/70.7%	96.7%	88.7%	8.0%	0	0	17/150
Aviation	DeepSeek-v4-Pro	SCIP	76.8%/70.0%	96.7%	86.7%	0.0%	0	0	15/150
Aviation	MIMO-V2.5-Pro	SCIP	78.9%/70.0%	94.0%	84.0%	0.0%	0	0	19/150
Architecture	qwen-plus	SCIP	95.3%/95.0%	96.0%	96.0%	0.0%	0	0	2/50
Architecture	DeepSeek-v4-Pro	SCIP	96.0%/100.0%	96.0%	96.0%	0.0%	0	0	2/50
Architecture	MIMO-V2.5-Pro	SCIP	92.0%/100.0%	96.0%	96.0%	0.0%	0	0	2/50

Table 7

Audit traceability of optimized decisions returned through different solver backends on the 200-task benchmark. Certificate coverage and valid-chain trace completeness are measured over task-backend rows; rule provenance validity is measured over the rule identifiers appearing in generated certificates.

Dataset	Method	Certificate coverage	Avg. rules per cert.	Rule provenance valid	Valid-chain trace complete
Aviation	CTHR+HiGHS	84.7%	1.906	100.0%	84.7%
Aviation	CTHR+CP-SAT/OR-Tools	95.3%	1.916	100.0%	95.3%
Aviation	CTHR+SCIP	96.7%	1.917	100.0%	96.7%
Architecture	CTHR+HiGHS	60.0%	2.233	100.0%	60.0%
Architecture	CTHR+CP-SAT/OR-Tools	88.0%	2.045	100.0%	88.0%
Architecture	CTHR+SCIP	88.0%	2.045	100.0%	88.0%

7.5. Audit Traceability

The preceding experiments evaluate whether the returned decisions are feasible and source-consistent. This section evaluates a different requirement: whether a decision can be audited after optimization. General-purpose solvers such as HiGHS, CP-SAT/OR-Tools, and SCIP can return feasible assignments, but they do not by themselves explain which source-linked rule chain was selected for the accepted decision. In CTHR, the solver is used only after the valid rule structures have been compiled into traceable feasible cells. Therefore, the audit certificate is a CTHR metadata object generated from the selected feasible cell and its rule chain, rather than from a solver-specific proof log.

Three backend replacements are evaluated over the same CTHR feasible cells: HiGHS, CP-SAT/OR-Tools, and SCIP. The evaluation is conducted on the new 200-task benchmark. *Certificate coverage* is defined as the fraction of task-backend rows for which a returned cell-level solution is accompanied by an audit certificate. *Rule provenance validity* checks whether each rule identifier listed in the certificate resolves to a rule record in the current rule library and whether that record contains non-empty provenance metadata such as document, section, or KG chunk identifiers. *Valid-chain trace completeness* checks whether the certificate lists all rule identifiers in the selected CTHR valid-rule chain used to compile the active feasible cell. Consequently, an accepted solution can be traced from the audit certificate to the valid rule chain, the rule-library entries, and the document or KG metadata stored in their provenance fields.

Table 7 shows that, whenever a backend returns a valid CTHR cell-level solution, the generated certificate preserves the rule-chain and provenance links. All certificate rule

identifiers in both domains resolve to provenance-bearing rule-library records. The main difference across backends is coverage: HiGHS covers 84.7% of aviation tasks but only 60.0% of architecture tasks, while CP-SAT/OR-Tools and SCIP cover 95.3% and 96.7% of aviation tasks and 88.0% of architecture tasks. This pattern is consistent with the cell-solver results: missing certificates mainly arise when the backend does not return a valid active-cell solution under the current numeric feasibility check, rather than from a broken provenance lookup. These results should be interpreted as a machine-checkable rule-chain and provenance linkage test, not as a solver-native proof log or a human audit-time study. They support the intended division of labor in CTHR: external solvers optimize over feasible cells, while the CTHR compilation layer supplies the auditable link from an optimized decision back to the KG-derived rules and source evidence.

7.6. Component Ablation

This experiment ablates key components of the CTHR modeling layer on the 200-task benchmark. The default configuration uses the predicted valid rules produced by the candidate-to-valid stage, the compiled rule-to-constraint templates, and the same backend family used in the main experiments. Each variant removes one component: candidate-to-valid profiling, scenario applicability, positive closure, negative resolution, relation-aware recovery, or compiled templates. The experiment reports solving success, formal and semantic constraint satisfaction, Rule-ID precision/recall. This is a component-ablation setting: variants remove selected modeling components rather than using hidden labels as inputs.

Table 8

Component ablation of the CTHR pipeline on the new aviation and architecture benchmarks. Rule P/R denotes per-task Rule-ID precision and recall; Exact denotes exact rule-set match. Aviation uses the SCIP backend version aligned with the main end-to-end evaluation.

Dataset	Variant	Solved	Formal CSR	Sem-CSR	Rule P/R
Aviation	CTHR default + SCIP	133/150	96.7%	88.7%	74.7% / 70.7%
Aviation	w/o Candidate-to-Valid Profiling	34/150	22.7%	22.7%	8.8% / 88.7%
Aviation	w/o Scenario Applicability	31/150	20.7%	20.7%	8.8% / 88.7%
Aviation	w/o Positive Closure	34/150	22.7%	22.7%	8.8% / 88.7%
Aviation	w/o Negative Resolution	34/150	22.7%	22.7%	8.9% / 91.7%
Aviation	w/o Relation-Aware Recovery	128/150	93.3%	85.3%	62.7% / 81.0%
Aviation	w/o Compiled Templates	113/150	83.3%	0.0%	74.7% / 70.7%
Architecture	CTHR default + SCIP	48/50	86.0%	72.0%	81.5% / 95.0%
Architecture	w/o Candidate-to-Valid Profiling	48/50	86.0%	72.0%	81.5% / 95.0%
Architecture	w/o Scenario Applicability	50/50	90.0%	76.0%	77.5% / 97.0%
Architecture	w/o Positive Closure	48/50	86.0%	72.0%	81.5% / 95.0%
Architecture	w/o Negative Resolution	48/50	86.0%	72.0%	80.1% / 95.0%
Architecture	w/o Relation-Aware Recovery	50/50	90.0%	76.0%	77.1% / 99.0%
Architecture	w/o Compiled Templates	48/50	80.0%	66.0%	81.5% / 95.0%

Table 9

Relation-level ablation of the CTHR rule-interaction layer. Each row removes one relation type from relation-aware recovery while keeping the SCIP backend fixed. Rule P/R denotes per-task Rule-ID precision and recall; Exact denotes exact rule-set match.

Dataset	Removed relation	Solved	Sem-CSR	Rule P/R	Exact
Aviation	None	133/150	88.7%	74.7% / 70.7%	132/150
Aviation	Scenario applicability	31/150	20.7%	8.8% / 88.7%	0/150
Aviation	Dependency	128/150	85.3%	62.7% / 81.0%	122/150
Aviation	Exclusion / branch	128/150	85.3%	62.7% / 81.0%	122/150
Aviation	Override	128/150	85.3%	62.7% / 81.0%	122/150
Aviation	Precedence	128/150	85.3%	62.7% / 81.0%	122/150
Aviation	Parameter propagation	128/150	85.3%	62.7% / 81.0%	122/150
Architecture	None	48/50	72.0%	81.5% / 95.0%	37/50
Architecture	Scenario applicability	50/50	76.0%	77.5% / 97.0%	33/50
Architecture	Dependency	48/50	72.0%	81.5% / 95.0%	37/50
Architecture	Exclusion / branch	48/50	72.0%	80.1% / 95.0%	35/50
Architecture	Override	48/50	72.0%	81.4% / 95.0%	37/50
Architecture	Precedence	48/50	72.0%	81.5% / 95.0%	37/50
Architecture	Parameter propagation	48/50	72.0%	81.5% / 95.0%	37/50

Table 8 shows that the component-level effects differ substantially across the two domains. In aviation, removing the early candidate-to-valid profile, scenario applicability, or closure/resolution components collapses the end-to-end solve rate to 20.7%–22.7% and reduces Rule-ID precision to below 9% (8.8%–8.9%), even though recall remains high (88.7%–91.7%). This confirms that broad aviation grounding contains many rules that are related to the procedure-design corpus but should not jointly define the current feasible cell. Removing relation-aware recovery is less destructive but drops the solve rate to 128/150, reducing Rule-ID precision to 62.7% while increasing recall to 81.0%. Removing compiled templates keeps the same valid-rule identifiers but drives Sem-CSR to 0.0%, isolating the constraint-compilation layer as essential for executable source semantics. The architecture ablation is less severe because the new architecture tasks and profile resolver already produce cleaner candidate sets, but removing scenario applicability or relation-aware recovery still increases recall up to 97.0%–99.0% at the cost of lower precision, indicating a looser rule set.

Table 9 reports the companion relation-level ablation. In aviation, removing scenario applicability is the strongest relation-level failure mode: the solve rate falls to 31/150,

Sem-CSR to 20.7%, Rule P/R to 8.8%/88.7%, and exact match to 0/150. Removing dependency, exclusion/branch, override, precedence, or parameter propagation gives an identical degradation pattern, with 128/150 solved tasks, 85.3% Sem-CSR, 62.7%/81.0% Rule P/R, and 122/150 exact matches. In architecture, removing scenario applicability and exclusion/branch mainly affects rule-set compactness: exact match drops from 37/50 to 33/50 and 35/50, respectively, whereas dependency, precedence, and parameter propagation remain marginal on this 50-task subset, preserving perfect alignment with the default baseline. Overall, the ablation supports a layered necessity claim: early profiling and scenario applicability control candidate noise, relation-aware recovery improves rule-set compactness, and compiled templates are needed to preserve source semantics at the constraint level.

7.7. Fairness and Controlled Scalability Checks

This section reports two controlled supplemental checks. First, all model-rule libraries are evaluated on the same task-level membership probes, including midpoint, boundary, near-boundary, random, and reference-feasible anchor points, with labels computed from hidden source-semantic references. The probe count is the total number of shared membership-test points, not the number of tasks or optimized solutions.

Table 10

Fairness and controlled candidate-noise checks. Panel (a) evaluates model-specific rule libraries on shared membership probes; panel (b) injects irrelevant candidate rules and reports aggregated robustness.

(a) Unified membership-probe evaluation							
Dataset	Model-rule library	Probe points	Semantic consistency	False accept	False reject	Accepted-point Sem-CSR	
Aviation	DeepSeek-v4-Pro	25,253	99.8%	0.0%	0.2%	98.5%	
Aviation	qwen-plus	25,253	100.0%	0.0%	0.0%	100.0%	
Aviation	Xiaomi MIMO	25,253	99.6%	0.0%	0.4%	100.0%	
Architecture	DeepSeek-v4-Pro	1,766	100.0%	0.0%	0.0%	100.0%	
Architecture	qwen-plus	1,766	99.9%	0.1%	0.0%	99.4%	
Architecture	Xiaomi MIMO	1,766	99.3%	0.7%	0.0%	96.4%	
(b) Controlled candidate-noise stress test							
Dataset	Candidate factor	Mean candidate rules	Mean compiled constraints	Mean semantic consistency	Minimum semantic consistency	Maximum false accept	Maximum false reject
Aviation	1x	2.173	48.649	99.8%	99.5%	0.0%	0.5%
Aviation	2x	4.347	49.056	99.6%	99.4%	0.0%	0.6%
Aviation	4x	8.693	50.031	99.3%	99.2%	0.0%	0.8%
Aviation	8x	17.387	52.151	98.9%	98.6%	0.0%	1.4%
Architecture	1x	1.907	11.333	100.0%	100.0%	0.0%	0.0%
Architecture	2x	3.813	11.493	100.0%	100.0%	0.0%	0.0%
Architecture	4x	7.627	11.593	99.9%	99.7%	0.0%	0.3%
Architecture	8x	15.253	11.953	100.0%	100.0%	0.0%	0.0%

Second, irrelevant candidate rules are injected to expand the candidate set to 2, 4, and 8 times its original size. These checks evaluate probe-design fairness and controlled candidate-noise robustness; they are not industrial-scale deployment claims.

Table 10 shows that the shared-probe evaluation does not favor a particular model-rule library. Semantic consistency ranges from 99.6% to 100.0% in aviation and from 99.3% to 100.0% in architecture; the largest false-accept rate is 0.7%. Under controlled candidate-noise injection, aviation remains above 98.9% mean semantic consistency at the 8x scale, and architecture remains between 99.9% and 100.0%. The largest observed false-reject rate is 1.4%.

8. Discussion and Limitations

CTHR should be interpreted as a provenance-preserving modeling layer, not as a replacement for optimization solvers. Its contribution is to construct source-consistent feasible cells and audit certificates before a backend solver searches for an objective-optimal point. The backend still matters for optimization strength, as reflected by the objective-gap results in Table 3. The solver-backend experiments therefore demonstrate representative adapters rather than universal solver coverage: additional SAT, SMT, MILP, NLP, global-optimization, or heuristic solvers would require their own encodings of the emitted feasible cells and objectives.

Some benchmark instances still do not produce a certified valid optimized decision. This limitation should be distinguished from the semantic preservation claim: the theory is conditional on correct valid-rule recovery and correct rule-to-constraint compilation, whereas the end-to-end experiments also include candidate filtering errors, rule-library imperfections, and backend-specific modeling limits. In the aviation benchmark, the remaining failures are mostly observed as source-semantically invalid returned decisions or missing backend solutions on a subset of tasks, despite the large reduction in invalid cases relative to flat and

native symbolic baselines. In the architecture benchmark, the number of such failures is smaller, but several tasks still fail because selected rule chains can be empty, overly broad, or not fully supported by a particular backend encoding. These cases indicate that CTHR supplies an auditable feasible-cell modeling interface for returned solutions, but it does not by itself guarantee that every source-grounded query is solvable by every backend. Improving fallback formulations, backend-specific encodings for indicator or count predicates, and calibration of candidate-to-valid recovery remain important future work.

The benchmark is a systematic two-domain task suite rather than a full industrial deployment. It supports controlled evaluation of source-grounded rule recovery, feasible-cell construction, decision validity, and certificate traceability, but broader deployment would require larger corpora and further expert review. The shared-probe and candidate-noise checks are controlled stress tests on this benchmark, not evidence of industrial-scale deployment. The clean split remains curated: synthetic aviation stress rules are excluded, architecture has limited multi-cell defeasible-chain stress, and some building-code requirements are encoded as executable semantic indicator or count predicates. Rule-library construction depends on KG-to-rule generator quality; downstream usability varies across LLM backbones and relies on exact or strong rule-ID alignment overlays. Finally, the architecture benchmark retains a small number of invalid cases, and certificate evaluation checks machine-readable traceability rather than human audit time. The current version also does not report an independent blind annotation study or inter-annotator agreement for the source-semantic references in this version.

9. Conclusion

This paper presents CTHR, a provenance-preserving KG-to-rule-to-feasible-cell modeling layer for auditable KG-driven optimization. CTHR resolves defeasible rule interactions, constructs valid rule structures, compiles them into source-semantics-preserving feasible cells, and returns audit certificates that link optimized decisions back to rule and document provenance.

Across the 200-task aviation and architecture benchmark, CTHR improves source-semantic decision validity over flat-rule modeling and supports multiple solver backends over the same feasible cells. The results show that auditable optimization depends not only on solver strength, but also on the upstream modeling step that decides which source rules jointly define the feasible region. Future work will extend KG-to-rule extraction, broaden the benchmark to additional regulated engineering domains, and reduce manual curation when natural-language design requests are converted into structured scenarios.

Funding

This research was supported by the National Natural Science Foundation of China (NSFC) under Grant No. 625B2123 and the Pioneer Fund of the System of Systems and Artificial Intelligence Key Laboratory of Sichuan Province under Grant No. SSAI202506.

A. Aviation Task Expert Review

To audit the quality of the aviation benchmark and its hidden reference labels, 100 aviation tasks were sampled from the 150-task aviation suite for expert review. The review was conducted by three flight-procedure design experts who had passed the CAAC Flight Procedure Design Basic Knowledge Examination. The audit was performed after benchmark construction and was used only to assess task and reference-label quality; it was not provided to CTHR, the candidate grounder, the profile resolver, or any solver backend.

Each expert scored every sampled task on five criteria using a 1–5 scale: 5 means that the task can be retained directly, 4 means that it is basically reasonable with only minor issues, 3 means that it is usable after minor revision, 2 means that it has clear issues and requires restructuring, and 1 means that it is unsuitable and should be removed or replaced. Table A.1 reports the mean scores across experts and reviewed tasks.

B. Proofs

B.1. Proof of Lemma L1 (Applicability Correctness)

Proof. The compiler constructs γ_k as the conjunction of: (a) grounded antecedents $\phi_r(\omega)$ for all $r \in \text{chain}(k)$; (b) dependency closures: for each $r \in \text{chain}(k)$, all $r' \in \text{dep}^+(r)$ (transitive dependency closure) must also be applicable; (c) internal exclusion consistency: no pair $r, r' \in \text{chain}(k)$

Table A.1

Expert-review protocol for the 100-task aviation benchmark sample. Scores are averaged over three certified flight-procedure design experts and the reviewed tasks.

ID	Evaluation question and review focus	Mean
Q1	Scenario realism: whether the scenario resembles a realistic flight-procedure design, regulation-review, or engineering-coordination task.	4.5
Q2	Reference-rule rationality: whether valid rules, excluded rules, priority or exception relations, and feasible-region constraints are reasonable.	4.0
Q3	Parameter and variable plausibility: whether units, ranges, variable names, and numerical magnitudes match domain practice.	4.2
Q4	Objective trade-off realism: whether the objectives reflect a meaningful operational trade-off rather than redundant goals.	4.3
Q5	Leakage avoidance: whether public inputs avoid exposing rule identifiers, reference answers, feasible-region labels, or certificate targets.	4.7

may satisfy $(r, r') \in \text{excl}$. Unsatisfiable guards are removed during Z3 normalization. This condition does not require an excluded alternative outside $\text{chain}(k)$ to be inapplicable. If two mutually exclusive alternatives are both applicable, they form separate source-level extensions and therefore separate feasible cells. The remaining guards are extensionally equivalent to source-level applicability by construction of items (a)–(c). $\square$

B.2. Proof of Lemma L2 (Override Correctness)

Proof. When a conflict class is resolved, the compiler identifies all applicable rules in the class and applies the defeat relation. For each $r_e \xrightarrow{\text{can_override}} r_b$ with both applicable: the compiler marks r_b as defeated and removes its consequent before polytope emission. Hence the emitted constraints for cell k contain c_{r_e} and exclude c_{r_b} , matching override semantics rather than flat conjunction. $\square$

B.3. Proof of Lemma L3 (Precedence Correctness)

Proof. Cross-document specificity, category specificity, and explicit superiority relations are compiled into a precedence order $>_\omega$ over conflicting rules. The compiler selects maximal non-defeated valid rule structures under this order using topological processing of the precedence DAG (well-founded by Assumption 1). Non-conflicting applicable rules are in disjoint conflict classes and are both retained. Therefore chain selection is extensionally equivalent to source-level precedence resolution. $\square$

B.4. Proof of Lemma L4 (Parameter Propagation Correctness)

Proof. By Assumption 2, the dep relation is acyclic, so $\eta(\omega)$ is uniquely determined by topological evaluation. The compiler performs symbolic substitution of $\eta(\omega)$ into each rule consequent $c_r(x, \eta) = \langle a_r(\eta), x \rangle \leq b_r(\eta)$ before affine canonicalization. Since $\eta(\omega)$ is unique and substitution is exact (no rounding), $c_r(x, \eta(\omega)) \iff \hat{c}_{r,\omega}(x)$ pointwise. $\square$

B.5. Proof of Lemma L5 (Geometry Correctness)

Proof. P_k is defined as the H-representation produced by stacking exactly the surviving instantiated inequalities of $\text{chain}(k)$ after defeasible resolution. Canonical affine rewrites preserve constraint meaning. Hence $x \in P_k \iff x$ satisfies all surviving instantiated consequents of $\text{chain}(k)$. $\square$

References

- [1] I. Horrocks, P. F. Patel-Schneider, H. Boley, S. Tabet, B. Grosz, M. Dean, SWRL: A semantic web rule language combining OWL and RuleML, W3C Member submission 21 (79) (2004) 1–31.
- [2] J. Z. Pan, G. Vetere, J. M. Gomez-Perez, H. Wu (Eds.), Exploiting Linked Data and Knowledge Graphs in Large Organisations, Springer International Publishing, Cham, 2017. doi:10.1007/978-3-319-45654-6.
- [3] P. Danti, D. Rolnick, J. Z. Kolter, DC3: A learning method for optimization with hard constraints, International Conference on Learning Representations (2021). URL <https://openreview.net/forum?id=V1ZHvXJ6dSS>
- [4] A. Navon, A. Shamsian, I. Achituv, H. Maron, K. Kawaguchi, G. Chechik, E. Fetaya, Learning the pareto front with hypernetworks, International Conference on Learning Representations (2021). URL <https://openreview.net/forum?id=NjF772F4ZZR>
- [5] L. de Moura, N. Björner, Z3: An efficient SMT solver, in: International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS), Springer, 2008, pp. 337–340. doi:10.1007/978-3-540-78800-3_24.
- [6] M. J. Sergot, F. Sadri, R. A. Kowalski, F. Kriwaczek, P. Hammond, H. T. Cory, The British Nationality Act as a logic program, Communications of the ACM 29 (5) (1986) 370–386. doi:10.1145/5689.5920.
- [7] C. Eastman, J.-m. Lee, Y.-s. Jeong, J.-k. Lee, Automatic rule-based checking of building designs, Automation in Construction 18 (8) (2009) 1011–1033. doi:10.1016/j.autcon.2009.07.002.
- [8] L. Moreau, P. Missier, PROV-DM: The PROV data model, W3C Recommendation, 30 April 2013 (2013). URL <https://www.w3.org/TR/prov-dm/>
- [9] E. Amador-Domínguez, E. Serrano, D. Manrique, Neurosymbolic system profiling: A template-based approach, Knowledge-Based Systems 287 (2024) 111441. doi:10.1016/j.knsys.2024.111441.
- [10] L. Some, W. Yang, M. Bain, B. Kang, A comprehensive survey on integrating large language models with knowledge-based methods, Knowledge-Based Systems 318 (2025) 113503. doi:10.1016/j.knsys.2025.113503.
- [11] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, X. Wu, Unifying large language models and knowledge graphs: A roadmap, IEEE Transactions on Knowledge and Data Engineering 36 (7) (2024) 3580–3599. doi:10.1109/TKDE.2024.3352100.
- [12] X. Dai, Y. Hua, T. Wu, Y. Sheng, Q. Ji, G. Qi, Large language models can better understand knowledge graphs than we thought, Knowledge-Based Systems 312 (2025) 113060. doi:10.1016/j.knsys.2025.113060.
- [13] R. Yang, J. Zhu, J. Man, L. Fang, Y. Zhou, Enhancing text-based knowledge graph completion with zero-shot large language models: A focus on semantic enhancement, Knowledge-Based Systems 300 (2024) 112155. doi:10.1016/j.knsys.2024.112155.
- [14] B. Jin, G. Liu, C. Han, M. Jiang, H. Ji, J. Han, Large language models on graphs: A comprehensive survey, IEEE Transactions on Knowledge and Data Engineering 36 (12) (2024) 8622–8642. doi:10.1109/TKDE.2024.3469578.
- [15] W. Huang, G. Zhou, M. Lapata, P. Vougiouklis, S. Montella, J. Z. Pan, Prompting large language models with knowledge graphs for question answering involving long-tail facts, Knowledge-Based Systems 324 (2025) 113648. doi:10.1016/j.knsys.2025.113648.
- [16] M. Hindi, L. Mohammed, O. Maaz, A. Alwarafy, Enhancing the precision and interpretability of retrieval-augmented generation (RAG) in legal technology: A survey, IEEE Access 13 (2025) 46171–46189. doi:10.1109/ACCESS.2025.3550145.
- [17] M. M. Hussien, A. N. Melo, A. L. Ballardini, C. S. Maldonado, R. Izquierdo, M. A. Sotelo, RAG-based explainable prediction of road users behaviors for automated driving using knowledge graphs and large language models, Expert Systems with Applications 265 (2025) 125914. doi:10.1016/j.eswa.2024.125914.
- [18] J. Lee, S. Ahn, D. Kim, D. Kim, Performance comparison of retrieval-augmented generation and fine-tuned large language models for construction safety management knowledge retrieval, Automation in Construction (2024) 105846. doi:10.1016/j.autcon.2024.105846.
- [19] Y. Wan, Z. Chen, Y. Liu, C. Chen, M. Packianather, Empowering LLMs by hybrid retrieval-augmented generation for domain-centric Q&A in smart manufacturing, Advanced Engineering Informatics 65 (2025) 103212. doi:10.1016/j.aei.2025.103212.
- [20] X. Xiong, H. Cai, H. Yu, B. Shen, P. Hu, DR-RAG: Domain-rule-based retrieval-augmented generation for aviation digital model design, Advanced Engineering Informatics 68 (2025) 103688. doi:10.1016/j.aei.2025.103688.
- [21] H. Knublauch, D. Kontokostas, Shapes constraint language (SHACL), W3C Recommendation, 20 July 2017 (2017). URL <https://www.w3.org/TR/shacl/>
- [22] Z. Liu, J. Li, Y. Huang, N. Cui, L. Pei, Knowledge-based natural answer generation via effective graph learning, Knowledge-Based Systems 316 (2025) 113288. doi:10.1016/j.knsys.2025.113288.
- [23] X. Zhou, S. Li, Y. Liu, W. Yang, C. Xiong, L. Hou, LLM-based domain-specific knowledge graph construction and hybrid retrieval-augmented generation, Journal of Manufacturing Systems 86 (2026) 1022–1038. doi:10.1016/j.jmsy.2026.04.023.
- [24] X. Chen, S. Zhang, X. Li, R. Ma, Q. Zhao, Dairygoatqa: A knowledge graph enhanced large language models approach for question answering in the dairy goat domain, Expert Systems with Applications 299 (2026) 130146. doi:10.1016/j.eswa.2025.130146.
- [25] H. Prakken, G. Sartor, The three faces of defeasibility in the law, Ratio Juris 17 (1) (2004) 118–139.
- [26] P. A. Bonatti, I. M. Petrova, L. Sauro, Defeasible reasoning in description logics: An overview on DLN, in: Applications and Practices in Ontology Design, Extraction, and Reasoning, Vol. 49 of Studies on the Semantic Web, IOS Press, 2020, pp. 178–193. doi:10.3233/SSW200043.
- [27] G. Brewka, T. Eiter, M. Truszczyński, Answer set programming at a glance, Communications of the ACM 54 (12) (2011) 92–103.
- [28] E. Giunchiglia, M. C. Stoian, S. Khan, F. Cuzzolin, T. Lukasiewicz, ROAD-R: The autonomous driving dataset with logical requirements, Machine Learning 112 (2023) 3261–3291. doi:10.1007/s10994-023-06322-z.
- [29] Z. Yang, A. Ishay, J. Lee, NeurASP: Embracing neural networks into answer set programming, in: Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, 2020, pp. 1755–1762. doi:10.24963/ijcai.2020/243.
- [30] G. Marra, M. Diligenti, F. Giannini, Relational reasoning networks, Knowledge-Based Systems 310 (2025) 112822. doi:10.1016/j.knsys.2024.112822.
- [31] Y. Chen, S. Mensah, J. Li, Learning distributed representations of knowledge that preserve deductive reasoning, Knowledge-Based Systems 293 (2024) 111635. doi:10.1016/j.knsys.2024.111635.
- [32] B. Amos, J. Z. Kolter, OptNet: Differentiable optimization as a layer in neural networks, in: Proceedings of the 34th International Conference on Machine Learning, Vol. 70 of Proceedings of Machine Learning Research, PMLR, 2017, pp. 136–145. URL <https://proceedings.mlr.press/v70/amos17a.html>
- [33] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, J. Z. Kolter, Differentiable convex optimization layers, Advances in Neural Information Processing Systems (NeurIPS) (2019).
- [34] J. Tordesillas, J. P. How, M. Hutter, RAYEN: Imposition of hard convex constraints on neural networks, arXiv preprint arXiv:2307.08336 (2023). doi:10.48550/arXiv.2307.08336.
- [35] D. Mahapatra, V. Rajan, Multi-task learning with user preferences: Gradient descent with controlled ascent in pareto optimization, in:

- Proceedings of the 37th International Conference on Machine Learning, Vol. 119 of Proceedings of Machine Learning Research, PMLR, 2020, pp. 6597–6607.
URL <https://proceedings.mlr.press/v119/mahapatra20a.html>
- [36] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, *IEEE Transactions on Evolutionary Computation* 6 (2) (2002) 182–197. doi:10.1109/4235.996017.
 - [37] K. Li, R. Chen, G. Fu, X. Yao, Two-archive evolutionary algorithm for constrained multi-objective optimization, *IEEE Transactions on Evolutionary Computation* 23 (2) (2019) 303–315. doi:10.1109/TEVC.2018.2855411.
 - [38] J. Liu, W. Hao, K. Cheng, G. Chen, X. Xie, CART: A traceable zero-shot planning framework for large language models with adaptive replanning, *Knowledge-Based Systems* 336 (2026) 115189. doi:10.1016/j.knosys.2025.115189.
 - [39] X. Yang, R. Wang, K. Li, W. Li, W. Huang, W. Liu, Large language model assisted meta-evolution for automated constrained optimization evolutionary algorithm design, *Expert Systems with Applications* 317 (2026) 131756. doi:10.1016/j.eswa.2026.131756.
 - [40] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhambri, L. P. Saldyt, A. B. Murthy, Position: LLMs can't plan, but can help planning in LLM-modulo frameworks, in: *Proceedings of the 41st International Conference on Machine Learning*, Vol. 235 of Proceedings of Machine Learning Research, PMLR, 2024, pp. 22895–22907.
URL <https://proceedings.mlr.press/v235/kambhampati24a.html>
 - [41] Topoteretes, Cognee: Build ai memory with a knowledge engine, <https://github.com/topoteretes/cognee>, open-source software, accessed May 20, 2026 (2025).